

리눅스(Linux)

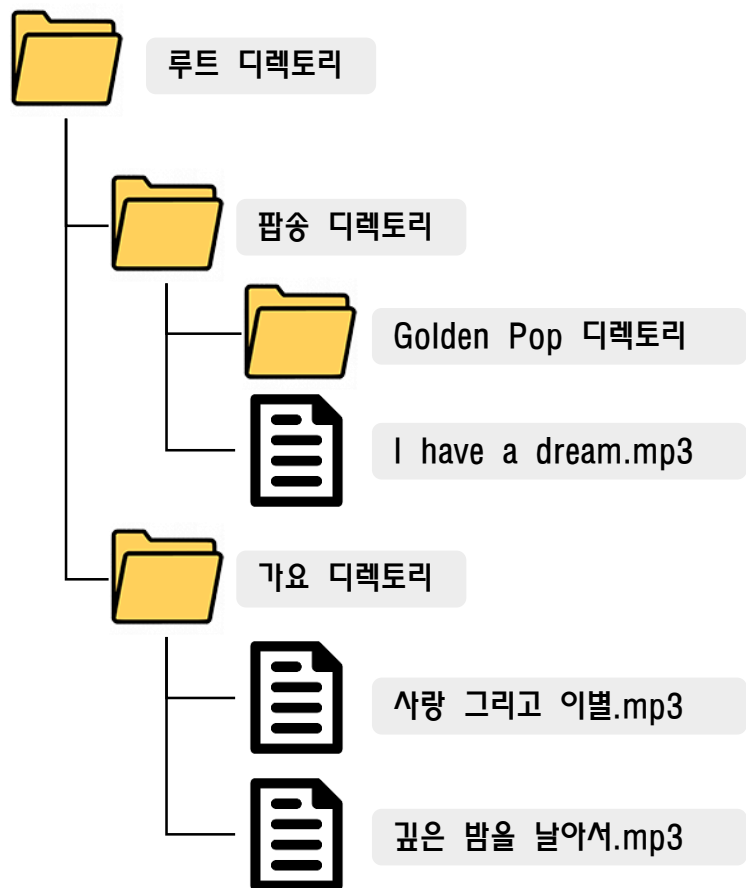
기본 명령어



≡ Basic Command

≡ 파일과 디렉토리

파일과 디렉토리에 대하여 간략히 살펴 보도록 하겠습니다.



파일

데이터를 저장하는 가장 기본 단위

디렉토리

파일과 다른 디렉토리를 담는 용기(Bottle) 역할

계층적 구조

루트 디렉토리 아래에 '팝송'과 '가요'라는 하위 디렉토리가 있으며, '팝송' 디렉토리 아래에는 'Golden Pop' 디렉토리와 파일 'I have a dream.mp3'가 있습니다. 이러한 구조를 통해 파일을 체계적으로 관리하고 쉽게 찾을 수 있습니다.

디렉토리는 파일 시스템의 중요한 구성 요소로, 파일을 논리적으로 구성하고 관리하는데 필수적인 역할을 합니다.

≡ Basic Command

≡ 파일(File)

파일은 데이터를 저장하는 가장 기본 단위입니다.

파일 예시

텍스트 파일

실행 파일

설정 파일

이미지 파일

파일의 특징

확장자는 필수 아님

파일의 종류는 내용과 권한으로 판단

실행 파일도 하나의 파일

≡ Basic Command

≡ 디렉토리(Directory)

디렉토리는 파일을 조직하고 관리하는 데 사용되는 특수한 유형의 파일입니다.

디렉토리

윈도우의 "폴더"와 같은 개념
파일을 체계적으로 정리하기 위해 사용

디렉토리의 특징

디렉토리는 파일을 포함할 수 있음
디렉토리 안에 또 다른 디렉토리 존재 가능
이 구조가 트리 구조(계층적 구조)를 형성

≡ Basic Command

≡ 리눅스 `Prompt` 구조

사용자가 터미널에서 명령을 입력할 수 있도록 표시하는 텍스트입니다.

prompt

항목	설명
root	로그인한 사용자 계정명
localhost	리눅스 시스템의 호스트명(/etc/sysconfig/network)
~	현재 작업 디렉토리 위치(~ 기호는 Home Directory를 의미)
#	관리자용 prompt
\$	일반 사용자 prompt

root@myserver:~#

ugcadman@myserver:~\$

ugcadman@myserver:~\$

①

②

③

① 접속 중인 계정 정보

② host 이름(컴퓨터 이름)

③ 현재 접속 위치

≡ 절대 경로와 상대 경로

≡ 경로(Path)란 무엇인가?

리눅스에서 파일을 찾으려면 반드시 "어디에 있는지"를 경로로 표현해야 합니다.

경로(Path)의 의미

파일이나 디렉토리의 위치 정보를 나타내는 주소

리눅스는 트리 구조(계층 구조)로 파일 시스템을 관리

모든 경로의 시작점은 루트 디렉토리(/)

≡ 절대 경로와 상대 경로

≡ 절대 경로(Absolute Path)

절대 경로는 "주소 전체를 모두 표현하는 방식"이라고 생각하시면 됩니다.

주요 특징

루트 디렉토리(/)부터 시작

서버 관리나 스크립트 작성 시에는 절대 경로를 많이 사용합니다.

위치가 바뀌어도 항상 동일한 결과를 보장합니다.

경로가 항상 /로 시작하므로 길어질 수 있음

명확하고 오류가 적음

사용 예시

```
/home/broadcast/kbs/1night_2days/hello.txt
```

≡ 절대 경로와 상대 경로

≡ 상대 경로(Relative Path)

상대 경로는 "지금 내가 있는 위치를 기준으로 어디로 갈지"를 표현하는 방식입니다.

주요 특징

현재 작업 중인 디렉토리 기준

/ 로 시작하지 않음

현재 위치에 따라 결과가 달라짐

사용 예시

(현재 위치: /home/broadcast)

kbs/1night_2days/hello.txt

주요 기호

기호	의미
.	현재 디렉토리
..	상위 디렉토리
./	현재 위치 기준
../	한 단계 위로 이동
~/	Home 디렉토리를 의미함

≡ 절대 경로와 상대 경로

≡ 절대 경로 vs 상대 경로 비교

시험에서는 "/"로 시작하면 절대 경로"라는 점만 정확히 기억해도 큰 도움이 됩니다.

주요 특징

구분	상대 경로	절대 경로
시작 위치	/(루트)	현재 디렉토리
현재 위치 영향	없음	있음
경로 길이	김	짧음
안정성	높음	낮음

≡ Basic Command

≡ `clear` 명령어

현재 작업중인 터미널의 화면 내용을 깨끗이 지웁니다.

터미널 창에 출력된 이전 명령어와 그 결과를 모두 지워 깔끔한 화면을 제공하는 데 유용하게 사용할 수 있습니다.

사용 구문

clear [옵션]

사용 옵션

항목	설명
-T type	터미널 타입을 지정합니다. 기본적으로 TERM 환경 변수를 사용합니다.

사용 예시

예시	설명
clear	터미널 화면을 깨끗하게 지웁니다.

≡ 디렉토리 관리

≡ `pwd(print working directory)

현재 작업중인 디렉토리의 절대 경로를 보여 줍니다.

pwd(print working directory) 명령어는 현재 작업 중인 디렉토리의 절대 경로를 출력합니다.

기본 옵션 -L은 논리적 경로를 출력하며, -P 옵션은 심볼릭 링크를 해제한 실제 경로를 출력합니다.

사용 구문

pwd [옵션]

사용 옵션

항목	설명
-L 또는 --logical	심볼릭 링크가 있는 경우 링크가 가리키는 경로를 출력합니다.
-P 또는 --physical	심볼릭 링크가 있는 경우 심볼릭 링크의 경로가 아니라 실제 경로를 출력합니다.

사용 예시

예시	설명
pwd	현재 작업 경로를 확인합니다.

≡ 디렉토리 관리

≡ `cd(change directory) 명령어

이동하고자 하는 디렉토리로 이동하기 위한 명령어입니다.

사용 구문

```
cd [ 인자값 ]
```

사용 옵션

옵션	설명
/	최상위 디렉토리를 의미 상위/하위 디렉토리를 위한 구분자
directory	이동하기 원하는 디렉토리
.	현재 작업 중인 디렉토리를 나타냅니다.
..	현재 디렉토리의 상위 디렉토리를 나타냅니다.
\$변수	변수에 지정된 정보를 이용하여 디렉토리 이동합니다.
~	로그인 된 사용자의 홈디렉토리로 이동합니다. 윈도우의 `내 문서`와 유사한 역할을 합니다.
~계정명	지정된 계정의 홈디렉토리로 이동합니다.

≡ 디렉토리 관리

≡ `ls`(디렉토리 목록 보기) 명령어

도스의 `dir`과 같은 역할을 하며, 해당 디렉토리에 있는 파일이나 디렉토리의 목록을 출력합니다.

`ls` 명령어는 파일 시스템을 탐색하고 파일 및 디렉토리의 정보를 확인하는 데 매우 유용한 도구입니다.

현재 디렉토리의 파일과 디렉토리의 목록을 출력하며, 다양한 옵션을 통해 출력 형식을 조정할 수 있습니다.

사용 구문

```
ls [ option ] [ directory / file ]
```

사용 옵션

옵션	설명
<code>-a, --all</code>	. 을 포함한 경로안의 모든 파일과 디렉토리 표시합니다.
<code>-l, --format=long</code>	지정한 디렉토리의 내용을 자세히 출력합니다.
<code>-d, --directory</code>	지정된 디렉토리의 정보 출력합니다.
<code>-n, --numeric</code>	파일 및 디렉토리 정보 출력시 UID, GID를 사용합니다.
<code>-F, --classify</code>	파일 형식을 알리는 문자를 각 파일 뒤에 추가합니다. ""(실행 가능한 파일), "/"(디렉토리), "->"(링크 파일로 오른쪽이 원본), "@", " ", "=", "NULL"
<code>-R, --recursive</code>	하위 경로와 그 안에 있는 모든 파일들도 같이 나열합니다.

≡ 디렉토리 관리

≡ `mkdir(make directory) 명령어

새로운 디렉토리를 생성하는 데 사용되며, 다양한 옵션을 통해 여러 가지 방식으로 디렉토리를 생성할 수 있습니다.

mkdir 명령어는 파일 시스템에서 디렉토리 구조를 설정할 때 매우 유용하며, 다양한 옵션을 통해 유연하게 사용할 수 있습니다.

사용 구문

```
mkdir [ -option ] [directory name]
```

사용 옵션

옵션	설명
-m, --mode	디렉토리 생성시 디렉토리의 기본 권한을 지정합니다.
-p, --parents	존재하지 않는 상위 경로의 디렉토리까지 강제로 생성해 줍니다.
--help	도움말 표시
--version	버전 정보 표시
-v 또는 -verbose	생성되는 각 디렉토리에 대해 메시지를 출력합니다.

≡ `rmdir(remove directory) 명령어

비어 있는 디렉토리를 삭제하기 위한 명령어입니다.

rmdir mydirectory 명령어는 비어 있지 않은 디렉토리를 삭제하려면 rm 명령어와 옵션을 사용해야 합니다.

파일 시스템에서 불필요한 빈 디렉토리를 제거하는 데 유용하며, 다양한 옵션을 통해 더욱 편리하게 사용할 수 있습니다.

사용 구문

이 명령어로 파일은 삭제할 수 없습니다.

하위 디렉토리가 존재하는 경우 삭제가 불가능합니다.

```
rmdir [ -option ] [ directory ]
```

사용 옵션

옵션	설명
-p, --parents	지정한 디렉토리와 그 부모 디렉토리가 비어 있으면 모두 삭제합니다.
-v 또는 --verbose	삭제되는 디렉토리에 대해 자세한 정보를 출력합니다.
--help	도움말 표시
--version	버전 정보 표시

≡ 파일 관리

≡ `touch` 명령어

파일의 시간 정보 변경 혹은 신규 파일을 생성하고자 할 때 사용합니다.

파일의 수정 날짜와 시간을 변경하거나, 새로운 파일을 생성하는 데 사용됩니다.

주로 빈 파일을 만들거나, 파일의 수정 시간을 업데이트할 때 사용됩니다.

사용 구문

```
touch [ option ] [ filename ]
```

사용 예시

파일이 존재하지 않을 경우 크기가 0인 새로운 파일을 생성하거나 기존에 존재하는 파일이면, 수정 시간을 변경해 줍니다.

```
# touch testfile
# ls -l
-rw-r--r--  1 root root      0  5월 24 10:14 testfile
```


≡ 파일 관리

≡ `touch` 명령어

신규 파일을 생성하거나 기존 파일의 시간 정보를 변경합니다.

파일이 존재하는 경우 파일의 생성 시간 및 최종 수정 시간을 변경해 줍니다.

```
# ls -l
-rw-r--r--  1 root root      0  5월 24 10:11 testfile
# touch testfile
# ls -l
-rw-r--r--  1 root root      0  5월 24 10:13 testfile
```

사용 옵션

옵션	설명
-a --time=atime	최근 파일 사용 시간만 업데이트합니다.
-c --no-create	지정한 파일이 없더라도 파일을 생성하지 않습니다.
-m --time=mtim	파일이 수정된 시간을 변경합니다.
-r [file] --reference [file]	지정한 파일의 시간으로 변경합니다.
-t [time]	현재시간 대신 지정한 시간으로 변경합니다.([[cc]yy]mmddhhmm[.ss])

≡ 파일 관리

≡ `cp(copy) 명령어

파일이나 디렉토리를 복사하는 명령어입니다.

여러 가지 옵션들을 사용하여 다양한 방식으로 파일을 복사할 수 있습니다.

사용 구문

```
cp [ -option ] [ sources ] [ target ]
```

사용 옵션

옵션	설명
-i, --interactive	복사 대상 파일이 있을 경우, 사용자에게 복사에 대한 실행 여부를 묻습니다.
-f, --force	복사 대상 파일이 있을 경우, 사용자에게 확인없이 강제로 복사합니다.
-r, -R, --recursive	디렉토리를 복사할 경우 하위 디렉토리 및 파일을 모두 복사합니다.
-v, --verbose	복사 진행 상태를 출력합니다.
-d, --no-dereference	복사 대상 파일이 심볼릭 파일이면, 심볼릭 정보를 그대로 유지한 상태로 복사합니다.
-p, --preserve	원본 파일의 소유주, 그룹, 권한, 시간 정보를 보존 하여 복사합니다.
-a, --archive(-dpr)	원본 파일의 속성, 링크 정보들을 그대로 유지하면서 복사합니다.

사용 예시

예시	설명
cp file1.txt /경로/새로운_위치/	파일을 다른 위치로 복사합니다.
cp file1.txt file2.txt /경로/새로운_위치/	여러 파일을 한 번에 복사합니다.
cp -r 디렉토리명 /새로운_위치/	디렉토리를 복사합니다.

≡ 파일 관리

≡ `mv(move) 명령어

파일이나 디렉토리를 이동하거나 이름을 바꿀 때 사용하는 명령어입니다.

사용 구문

```
mv [ -option ] [ sources ] [ target ]
```

사용 옵션

옵션	설명
-i, --interactive	기본적으로 .bashrc에 alias되어 있는 옵션으로, 이동할 위치에 동일한 파일이 있을 경우 사용자에게 확인
-u, --update	이동할 파일이 이동할 위치에 있는 파일보다 최근 파일일 경우에만 이동합니다.
-b, --backup	대상 파일이 이미 있어, 지워지는 것을 대비해 백업 파일을 생성해 줍니다.
-f, --force	대상 파일이 이미 있어도 사용자에게 어떻게 처리할지를 묻지 않습니다.
-v, --verbose	파일 을 옮기는 과정을 자세하게 보여 줍니다.
-S, --suffix	-b 옵션을 이용하여 백업할 경우 백업파일에서 사용할 파일 이름의 꼬리 문자를 지정합니다.

사용 예시

예시	설명
<code>mv file.txt /home/user/documents</code>	현재 디렉토리에 있는 file.txt를 /home/user/documents 디렉토리로 이동합니다.

≡ 파일 관리

≡ `rm` 명령어

파일 및 디렉토리를 삭제하기 위한 명령어 입니다.

잘못 사용하는 경우 중요한 데이터를 잃을 수 있으므로 주의해서 사용해야 합니다.

사용 구문

```
rm [ -option ] [ directory / file ]
```

사용 옵션

옵션	설명
-f, --force	파일/디렉토리 삭제시 사용자에게 어떻게 처리할지 물어보지 않습니다.
-r, -R, --recursive	일반 파일이면 그냥 지우고, 디렉토리면 디렉토리를 포함한 하위 경로와 파일을 모두 지웁니다.
-v, --verbose	각각의 파일 지우는 정보를 자세하게 보여 줍니다.
--version	버전 정보를 보여 줍니다.

사용 예시

예시	설명
rm -rf drama31	디렉토리를 삭제하려면 -r 옵션이 필요합니다.
rm -rf on80/ mbc11/	여러 디렉토리를 동시에 삭제할 수 있습니다.

≡ `cat(concatenate) 명령어

텍스트 파일 내용을 표준 출력 장치로 출력하는 명령어 입니다.

`cat`은 "concatenate"의 줄임말이고, 하나 이상의 파일을 인자로 받을 수 있으므로, 여러 파일의 내용을 연속해서 출력할 수 있습니다. 또한 파일의 내용을 터미널에 출력하는 것 외에도 파일을 결합하여 새로운 파일을 만드는 데에도 사용될 수 있습니다.

사용 구문

```
cat [ option ] [ file ]
```

사용 예시

```
$ cat /etc/passwd # 파일 내용 출력하기
```

```
$ cat /etc/passwd > /testfile # 기존의 파일 내용을 다른 파일로 입력하기
```

```
$ cat >> /testfile # 기존 파일에 내용 추가하기
```

사용 옵션

옵션	설명
-b	공백을 제외한 파일 내용의 줄의 맨 앞에 1부터 시작하는 줄 수를 보여 줍니다.
-E	각 줄의 끝에 \$로 끝줄을 나타냅니다.
-n	모든 줄의 앞에 1부터 시작하는 줄 수를 보여 줍니다.
-s	공백의 줄 수를 하나이상 보여 주지 않습니다.
-T	탭 문자를 "시"로 보여 줍니다.
-v	개행 문자와 탭 문자를 제외한 시와 M 표시를 보여 줍니다.

≡ 파일 관리

≡ `more` 명령어

긴 텍스트 파일을 한 페이지씩 화면에 나누어 보여 주는 도구입니다.

긴 텍스트 파일을 한 페이지씩 출력해서 읽기 쉽게 해줍니다.

출력이 스크롤되지 않고 멈추기 때문에, 내용을 천천히 검토할 수 있습니다.

[Enter] 키를 누르면, 한줄씩 출력이 되고, [SpaceBar] 키를 누르면 한 화면씩 출력이 됩니다.

종료하려면 `q` 키를 누르면 됩니다.

사용 구문

```
more [옵션] 파일명
```

사용 예시

```
# 파일 내용을 한 페이지씩 보기
```

```
more myfile.txt
```

```
# 여러 파일을 차례로 보기
```

```
more file1.txt file2.txt
```

```
# 명령어 출력 결과를 more로 넘기기
```

```
ls -l | more
```

≡ 파일 관리

≡ less 명령어

텍스트 파일을 한 화면씩 읽어 보며 탐색할 수 있도록 도와 주는 페이지 뷰어(pager)입니다.

more 명령어와 비슷하지만, 훨씬 많은 기능을 제공합니다.

일반적으로 긴 텍스트 파일이나 명령어 출력 결과를 스크롤하면서 볼 때 사용됩니다.

사용 구문

```
less [옵션] 파일명
```

≡ 파일 관리

≡ `find` 명령어

파일 시스템을 검색하여 파일이나 디렉토리를 찾는 데 사용됩니다.

검색 조건으로는 파일 이름, 파일 타입, 크기, 수정 시간, 사용자, 그룹, 권한 등이 있습니다.

검색 결과에 대해 명령어를 실행하거나, 결과를 출력하거나, 삭제할 수 있습니다.

사용 구문

```
find [검색 시작 경로] [옵션] [검색 조건] [후속 작업]
```

사용 옵션

옵션	설명
파일 이름으로 검색	<code>find <경로> -name <파일명></code>
Access time 이 n 일 보다 작은	<code>find <경로> -atime -n</code>
Access time 이 n 일 보다 큰 파일	<code>find <경로> -atime +n</code>
test 파일 이후에 수정된 모든 파일을 찾기	<code>find /home/ -newer test</code>
명령 수행	<code>find . -name "test*" -exec rm {} \;</code>
root 권한으로 실행되는 파일 찾기	<code>find . -user root -perm +4000 2> /dev/null</code>

사용 예시

예시	설명
<code>find /var/log -name "*.log"</code>	/var/log 경로에서 확장자가 .log인 파일을 찾습니다.
<code>find /home/user -atime -3</code>	/home/user 경로에서 최근 3일 이내에 접근한 파일을 찾습니다.
<code>find /backup -size +500k -size -10M</code>	/backup 경로에서 크기가 500KB 이상 10MB 미만인 파일을 찾습니다.

≡ 파일 관리

≡ `grep` 명령어

텍스트에서 **정규 표현식으로 특정 패턴을 검색**하고, 해당 패턴이 포함된 줄을 출력하는 명령어입니다.

GREP = Global Regular Expression Print

주로 로그 파일 분석, 파일 내용 검색, 또는 텍스트 데이터를 처리할 때 유용하게 사용됩니다.

사용 구문

패턴 : 검색할 문자열이나 정규 표현식입니다.

파일 : 검색할 파일을 지정합니다. 여러 개의 파일도 지정할 수 있습니다.

grep [옵션] '패턴' [파일...]

사용 예시

예시	설명
grep -i 'hello' file.txt	대소문자를 구분하지 않고 검색하므로, hello, Hello, HELLO 등을 모두 찾습니다.
grep -v 'error' file.log	패턴을 포함하지 않는 줄을 출력하므로, error가 포함되지 않은 줄을 출력합니다.
grep -r 'pattern' /path/to/directory	디렉토리 및 그 하위 디렉토리에서 파일을 재귀적으로 검색합니다.
grep -l 'success' *.log	확장자가 log 파일 중에서 success 글자가 포함된 항목만 출력합니다.
grep -n 'pattern' filename	패턴이 포함된 줄 번호와 함께 출력합니다.
grep -c 'pattern' filename	패턴이 포함된 줄의 개수를 출력합니다.
grep -H 'pattern' filename	파일 이름과 함께 결과를 출력합니다.
grep -w 'word' filename	단어 단위로 검색합니다.(패턴이 다른 단어의 일부로 포함되지 않음)
grep -x 'exact pattern' filename	줄 전체가 패턴과 일치하는 경우에만 출력합니다.

≡ `|` 기호(pipe)

| 기호는 파이프(pipe)를 나타내며, 한 명령어의 출력을 다른 명령어의 입력으로 전달하는 데 사용됩니다.

여러 개의 명령어를 연결하여 파이프라인을 형성할 수 있습니다.

각 명령어는 그 출력 결과를 다음 명령어의 입력으로 전달하므로, 복잡한 작업을 효율적으로 처리할 수 있습니다.

사용 예시

파일 목록 중에서 "file"이라는 글자가 포함된 목록만 조회합니다.

```
ls | grep "file"
```

".txt" 확장자를 가진 파일만 grep 명령어로 필터링합니다.

```
ls -l | grep ".txt"
```

ls와 sort를 사용하여 파일 목록 정렬

```
ls | sort
```

"/etc/passwd"라는 파일 내에서 "bluesky"이라는 글자가 있는 라인만 보여 줍니다.

```
tail /etc/passwd | grep bluesky
```

≡ `리다이렉션`(`redirection`)

명령어의 입력과 출력을 파일로 저장하거나 다른 명령어의 입력으로 연결하는 데 사용됩니다.

리눅스의 리다이렉션 기능은 명령어의 입출력을 유연하게 처리할 수 있게 해줍니다.

이를 통해 파일 입출력, 오류 처리, 명령어 연결 등의 작업을 효율적으로 수행할 수 있습니다.

연산자

항목	설명
>	파일로 출력(기존 파일을 덮어씁니다)
>>	파일로 출력(기존 파일에 내용을 추가합니다)
<	입력 리다이렉션(파일에서 입력을 받습니다.)

사용 예시

예시	설명
<code>echo "Hello, World!" > file.txt</code>	"Hello, World!"를 file.txt 파일에 기록합니다.
<code>echo "여러분~~안녕하세요" >> file.txt</code>	"여러분~~안녕하세요"를 file.txt 파일의 끝에 추가합니다.

≡ 파일 관리

≡ `awk` 명령어

파일이나 입력 스트림에서 열(column) 단위 데이터 처리, 패턴 검색, 보고서 생성 등에 사용됩니다.

awk는 자체적인 간단한 프로그래밍 언어를 내장하고 있어서 다양한 조건과 로직을 적용할 수 있습니다.

사용 구문

`awk '패턴 { 액션 }' 파일명`

패턴: 적용할 조건(예: 특정 문자열 포함, 특정 열의 값 등)

액션: 조건을 만족하는 행에 대해 수행할 작업

파일명: 처리할 텍스트 파일 또는 입력 스트림

사용 예시

특정 열 출력

`awk '{ print $2, $3 }' data.txt` # 각 줄의 두 번째, 세 번째 필드(컬럼) 정보를 출력

조건문 사용

`awk '$3 > 80 { print $1, $3 }' score.txt` # 세 번째 필드가 80보다 큰 행만 출력

필드 구분자 지정(예: CSV 파일)

`awk -F ',' '{ print $1, $3 }' file.csv`

≡ 파일 관리

≡ `diff` 명령어

두 파일의 차이점을 비교해서 보여 주는 Linux/Unix 계열 시스템의 명령어입니다.

아래에 diff 명령어의 기본 구조, 출력 형식, 주요 옵션, 실용 예제 등을 포함해 세부적으로 설명해 드리겠습니다.

사용 구문

```
diff [옵션] 파일1 파일2
```

사용 예시

일반 비교

```
diff file1.txt file2.txt
```

변경된 부분을 유닉스 스타일로 보기

```
diff -u file1.txt file2.txt
```

```
# @@ -1,3 +1,3 @@
```

```
# -Hello World
```

```
# +Hello Linux
```

```
# This is a test.
```

```
# Bye!
```

좌우 비교 형식으로 보기

```
diff -y file1.txt file2.txt
```

```
# Hello World           | Hello Linux
```

```
# This is a test.       = This is a test.
```

```
# Bye!                   = Bye!
```

≡ 파일 관리

≡ `file` 명령어

파일의 종류(type) 를 확인할 수 있는 명령어입니다.

단순히 파일의 확장자가 아니라, 실제 파일의 내용(header, magic number 등) 을 분석해서 어떤 유형의 파일인지 판별해줍니다.

사용 구문

```
file [옵션] 파일명
```

사용 예시

```
# 확장자 없이 파일 종류 알아보기
```

```
file unknown_file
```

```
# MIME 타입 확인
```

```
file -i photo.jpg
```

```
# 결과: photo.jpg: image/jpeg; charset=binary
```

```
# 압축 파일 내부 확인
```

```
file -z compressed.gz
```

```
# 파일 리스트에서 일괄 확인
```

```
file -f file_list.txt
```

```
# file_list.txt 안에 검사할 파일 이름을 한 줄씩 기록
```

≡ 파일 관리

≡ `which` 명령어

명령어(실행 파일)의 경로를 알려주는 도구입니다.

일반적으로 어떠한 명령어가 실제로 시스템의 어디에 설치되어 있는지를 확인할 때 사용됩니다.

사용 구문

`which [옵션] 명령어이름`

사용 예시

```
which python
# /usr/bin/python
# 이는 python 명령어가 /usr/bin/python 위치에 `존재한다`는 의미입니다.
```

≡ 자동 완성 기능

≡ `자동 완성 기능(auto tab)

키보드의 tab 키를 눌러서 파일의 이름이나 디렉토리 이름을 자동으로 채워 주는 기능입니다.

자동 완성

예를 들어서 /docker/comp 라고 입력한 이후에 tab 키를 누르면 현재의 디렉토리에서 comp로 시작하는 디렉토리를 검색합니다.
일치하는 것이 하나만 존재하면 자동 입력이 됩니다.
2개 이상이 존재하면 목록이 표시됩니다.

리눅스(Linux)

사용자 계정 관리



≡ 사용자 계정 관리

≡ 사용자와 그룹

리눅스는 1대의 컴퓨터에 여러 명이 동시 접속 가능한 다중 사용자 시스템입니다.

기본적으로 root라는 슈퍼 유저가 존재하고, 시스템의 거의 모든 기능들을 실행할 수 있는 권한이 있습니다.

그리고, 시스템에 접속할 수 있는 사용자를 생성할 수 있는 권한이 있습니다.

사용자 계정/그룹 관리

위치	의미
/etc/passwd	사용자 계정 정보의 저장 장소
/etc/shadow	안전하게 보호될 사용자 계정 정보가 들어 있는 파일입니다.
/etc/group	계정이 속한 그룹 ID 및 그룹 목록들에 대한 정보
/etc/gshadow	그룹의 암호 정보 저장 장소
/home	사용자별 개별 홈 디렉토리가 생성되는 기본 디렉토리
/etc/login.defs	사용자 계정의 기본 설정을 정의하는 구성 파일입니다. 사용자 계정의 생성, 로그인, 패스워드 정책, 홈 디렉토리, 사용자 ID 및 그룹 ID 등의 기본 동작을 제어합니다. 주로 useradd, userdel, passwd 등의 명령어가 이 파일에서 설정한 값을 참조합니다.
/etc/default/useradd	새 사용자 계정이 생성될 때 자동으로 적용되는 기본값들이 저장되어 있습니다. 예를 들어, 사용자 계정을 만들 때의 기본 셸, 홈 디렉토리 위치, UID 및 GID 범위 등이 설정될 수 있습니다.
/etc/skel	새로운 사용자 계정을 생성할 때 사용되는 기본 템플릿 파일들을 포함하는 디렉토리입니다. 이 디렉토리에 저장된 파일들은 새로운 사용자가 생성될 때 해당 사용자의 홈 디렉토리에 복사됩니다.

≡ 사용자 계정 관리

≡ `/etc/passwd`

시스템에 있는 모든 **사용자 계정 정보**를 저장하고 있는 파일입니다.

사용 형식

사용자명:암호:사용자ID(UID):그룹ID(GID):사용자 정보:홈 디렉토리:기본 셸

사용 예시

someuser:x:500:500:: /home/someuser:/bin/bash

필드	의미
someuser	사용자 계정 이름으로 로그인시 사용됩니다.
x	사용자에게 부여된 비밀번호로써, 보안을 위해 암호의 해시가 저장되지 않고, x로 표시됩니다. 실제 암호에 대한 해시는 /etc/shadow 파일에 저장됩니다.
500	사용자 계정 id를 고유하게 식별하는 숫자입니다. 루트 사용자(0), 시스템 계정(시스템 프로세스와 관련된 사용자, 1~999), 일반 사용자(1000 이상)
500	이 사용자에게 대한 숫자로 표현되는 주 그룹 id(GID)로써, 기본 값은 사용자 계정 id와 동일한 값입니다. /etc/group 파일에서 그룹 ID에 대한 추가 정보를 확인할 수 있습니다.
NULL	사용자에게 대한 부가적인 정보(이름, 전화번호 등)를 저장합니다. 보통 비어 있는 값이거나 관리자가 사용자의 실제 이름이나 직책 같은 정보를 넣을 수 있습니다.
/home/someuser	사용자 계정에 개별 홈디렉토리 위치로써, 개인 파일이 저장되는 곳입니다.
/bin/bash	사용자가 시스템에 로그인시 사용하는 기본으로 사용하는 셸 프로그램입니다. 예시: /bin/bash, /bin/zsh, /sbin/nologin.

≡ /etc/shadow

사용자 계정의 암호 해시 및 계정 잠금 정보를 저장해 놓은 시스템 파일입니다.

"Aging 정보", /etc/shadow 파일에서 관리하는 비밀번호 및 계정의 유효 기간과 변경 주기를 설정하는 항목을 말합니다.

aging 정보란, 사용자 계정의 비밀번호가 언제 만료되는지, 언제 변경해야 하는지 등 "비밀번호 사용 기간"과 관련된 정보를 의미합니다.

즉, 'password aging' 또는 'password aging information'이라고 부르며, 시스템 보안을 강화하기 위해 사용되는 중요한 기능입니다.

사용 예시

```
someuser:!!:14349:0:99999:7:: :
```

필드	의미
someuser	사용자 계정 명을 의미합니다.
!!	사용자에게 부여된 기본 암호(인증 불가)로써 암호화 되어 있습니다.
14349	someuser 계정에게 부여된 암호 마지막 부여 일자 유닉스 생성된 이후 1970.1.1을 기준으로 지난 일수를 표현합니다.
0	someuser 계정의 암호를 변경할 수 있는 최소 기간
99999	someuser 계정에게 부여된 암호를 변경없이 사용할 수 있는 유효 기간(유지 가능 최대 일수)
7	만료일 지정시 만료 경고 일수
Null	계정 만료 일자와 비활성화 일수
Null	계정 만료 날짜(기본 값은 지정되어 있지 않습니다.)

≡ 사용자 계정 관리

≡ /etc/group

시스템에 정의된 모든 그룹과 각 그룹에 속한 사용자들을 관리하기 위한 파일입니다.

이 파일은 보안상의 이유로 일반적으로 루트 사용자만 읽을 수 있습니다.

/etc/group 파일과 함께 그룹 관련 정보를 저장하지만, /etc/gshadow는 그룹 비밀번호나 특권 정보를 포함하여 추가적인 보안 정보를 관리합니다.

사용 형식

```
group_name:x:GID:user_list
```

필드	의미
group_name	그룹의 이름
x	암호 필드. 일반적으로 x로 표시되며, 암호가 /etc/gshadow 파일에 저장됨을 의미합니다.
GID	그룹 ID. 시스템에서 그룹을 식별하는 고유 번호.
user_list	그룹 구성원 모두의 사용자 이름으로 콤마 기호로 구분이 됩니다.

사용 예시

```
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,ugcadman
```

≡ 사용자 계정 관리

≡ /etc/gshadow

그룹 비밀번호 및 관련 정보를 관리하는 파일입니다.

이 파일은 보안상의 이유로 일반적으로 루트 사용자만 읽을 수 있습니다.

/etc/group 파일과 함께 그룹 관련 정보를 저장하지만, /etc/gshadow는 그룹 비밀번호나 특권 정보를 포함하여 추가적인 보안 정보를 관리합니다.

사용 형식

그룹명:암호:관리자:사용자

사용 예시

```
adm:*::root
developers:!::alice,bob
staff:$6$randomsalt$encryptedpassword:manager:john,doe
```

항목	설명
adm	비밀 번호 필드는 *로 설정되어, 그룹 비밀번호를 사용할 수 없습니다. 관리자는 없으며, 사용자는 root입니다.
developers	비밀 번호가 !로 설정되어 있어 비활성화 상태입니다. 관리자는 없으며, alice와 bob이 이 그룹에 속합니다. 만약 비밀번호가 !!이면 `설정되어 있지 않음`을 의미합니다.
staff	비밀 번호가 암호화된 형태로 저장되어 있으며, 이 비밀번호를 통해 그룹에 접근할 수 있습니다. 관리자는 manager이고, 사용자는 john과 doe입니다.

≡ 사용자 계정 관리

≡ 기타 파일

지금까지 명시한 파일 이외에 다음과 같은 파일들이 있습니다.

이 파일은 보안상의 이유로 일반적으로 루트 사용자만 읽을 수 있습니다.

/etc/group 파일과 함께 그룹 관련 정보를 저장하지만, /etc/gshadow는 그룹 비밀 번호나 특권 정보를 포함하여 추가적인 보안 정보를 관리합니다.

파일	설명
/etc/login.defs	shadow password suite configuration 해당 계정의 비밀 번호 유효 기간 계정 생성시 자동으로 할당되는 UID/GID 범위
/etc/default/useradd	Default values for account creation /etc/passwd 파일에 생성되는 대부분의 기본 값 SKEL 디렉토리 경로
/etc/skel	Directory containing default files 홈 디렉토리가 생성 되면서 복사될 기본 파일들이 들어 있습니다. 만약 계정이 생성된 후 생성되는 계정의 사용자들에게 공지할 내용이 있는 경우, /etc/skel 디렉토리에 내용을 포함해 둘 수 있습니다.



/etc/skel 디렉토리



사용자 홈 디렉토리

≡ 기타 파일

/etc/login.defs 파일에 대하여 살펴 보도록 하겠습니다.

/etc/login.defs 파일은 useradd 명령어가 새로운 계정을 생성할 때 반드시 참조하는 파일입니다.
아래의 예시에 주석을 작성되어 있으므로 참고 바랍니다.

login.defs 파일

```
PASS_MAX_DAYS    99999 //패스워드 변경 없이 사용할 수 있는 최대 일자
PASS_MIN_DAYS    0 //패스워드 변경 없이 사용할 수 있는 최소 일자
PASS_MIN_LEN     5 //패스워드의 최소 바이트 수
PASS_WARN_AGE    7 //패스워드 변경 경고 메시지 보내는 기간(유효 기간으로부터 x일)
UID_MIN          500 //생성할 수 있는 UID 시작 번호
UID_MAX          60000 //생성할 수 있는 최대 UID 번호
GID_MIN          500 //새롭게 생성되는 그룹의 GID 시작 번호
GID_MAX          60000 //새롭게 생성되는 그룹에서 지정할 수 있는 최대 GID 번호
CREATE_HOME      yes // 홈 디렉토리를 생성할 것인가의 여부
```


≡ 기타 파일

/etc/default/useradd 파일에 대하여 살펴 보도록 하겠습니다.

이 파일을 "useradd의 기본 파일" 이라고 부릅니다.

useradd 명령어로 계정을 생성할 때 어떤 환경과 어떤 파일들을 참조하여 새로운 계정을 생성할 것인가에 대하여 정의를 하고 있는 파일입니다.

useradd 파일

```
GROUP=100 //기본 등록 그룹의 GID
HOME=/home //생성될 홈 디렉토리의 위치
INACTIVE=-1 //패스워드 종료일 이후의 유효(기간) 여부 설정(0, -1, 1)
EXPIRE= //계정 종료 일자 지정
SHELL=/bin/bash //기본 사용 셸 지정
SKEL=/etc/skel //홈 디렉토리에 복사할 기본 환경 파일의 위치
CREATE_MAIL_SPOOL=yes //메일 저장 파일 여부
```

≡ 사용자 계정 관리

≡ `useradd` 계정 생성 명령

새로운 사용자를 생성할 때 사용하는 명령어입니다.

원하는 계정을 생성할 때 사용하는 리눅스 관리자 명령어입니다.

이 명령어는 관리자용 명령어이므로 일반 사용자는 사용이 불가능합니다.

계정 생성 관련 업무

/etc/passwd 파일에 사용자의 계정(ID 및 패스워드) 생성

/etc/shadow 파일에 계정 패스워드 등록

/etc/group 파일에 사용자의 그룹 생성(그룹 정보를 담고 있는 파일)

/home 디렉토리에 사용자의 홈 디렉토리 생성

ftp 사용을 위한 ftp 사용 환경 설정

메일 사용을 위한 메일 환경 설정

사용자의 디렉토리 및 파일 소유권 변경 및 확인

DB 사용을 위한 환경 설정 응용 및 응용 프로그램 사용을 위한 환경 설정

기타 응용 소프트웨어 사용을 위한 설정 등등

≡ `useradd` 명령어

새로운 사용자를 생성할 때 사용하는 명령어입니다.

사용 형식

```
useradd [Option] [Argument] ... [Account_Name]
```

필드별 옵션	의미	필드별 옵션	의미
Null	사용자 계정명은 직접 지정	-b	계정 생성시 생성되는 홈 디렉토리의 기본 생성 위치 변경
-p	사용자의 비밀번호 생성(MD5_Crypt)	-f	계정 만료 일자에 대한 Inactive Days 변경
-u	사용자의 uid 정보를 임의로 변경	-g	계정 생성시 지정되는 기본 그룹 변경
-g	사용자의 기본그룹 지정시 사용	-s	계정의 기본 로그인 쉘 변경
-e	계정의 기본 만료 일자 변경(이 날짜까지만 사용하도록 하겠습니다.)	-s	사용자의 로그인 쉘 변경
-c	사용자에게 필요한 설명(comment) 부여	-r	사용자의 uid 정보를 1~499번 사이의 값으로 생성
-m	사용자의 홈 디렉토리를 생성해 줍니다.	-d	사용자의 기본 홈 디렉토리 변경
-D	기본 환경 설정 변경하기	-o(non-unique)	동일한 uid값을 갖는 계정 생성

≡ 사용자 계정 관리

≡ `useradd` 명령어

useradd -D 옵션은 useradd의 기본 환경 설정을 변경하는 역할을 합니다.

사용 형식

```
useradd -D [-g 기본그룹] [-b 기본홈디렉토리] [-g default_inactive] [-e 기본계정종료일] [-s 기본사용셸]
```

/etc/default/useradd

다음은 cat /etc/default/useradd 명령어를 실행시켜 나온 결과입니다.

```
GROUP=100 # -g
HOME=/home # -b
INACTIVE=-1 # -f
EXPIRE= # -e
SHELL=/bin/bash # -s
SKEL=/etc/skel # 옵션 없음
CREATE_MAIL_SPOOL=yes # 옵션 없음
```

좌측 예시에서 # 기호 이후의 항목은 실제 존재하는 명령어가 아닙니다.

명령어로 사용하고자 할 때 사용되는 옵션을 명시해 둔 것입니다.

예를 들어서 GROUP 항목은 "-g" 라는 옵션을 이용하여 변경시킬 수 있음을 의미합니다.

more example

```
# useradd -D -s /bin/ksh : 셸의 기본 값을 변경합니다.
```

≡ 사용자 계정 관리

≡ `usermod` 명령어

사용자 계정의 속성을 수정하는 데 사용되는 명령어입니다.

사용 형식

```
usermod [Option] [Argument] ... [Account_Name]
```

필드별 옵션	의미
-a	-G와 함께 사용되며, 기존 그룹에 사용자를 추가해 줍니다.
-l	사용자의 로그인 이름(계정명)을 변경합니다.
-p	사용자의 비밀번호 변경(MD5_Crypt)
-u	사용자의 uid 정보를 임의로 변경
-g	사용자의 기본 그룹을 변경하고자 할 때 사용합니다.
-G	사용자에게 추가 그룹 지정(-a 옵션과 함께 사용)
-c	사용자 계정에 대한 설명을 변경합니다.
-d	사용자의 기본 홈 디렉토리를 변경(-m 옵션과 함께 사용)합니다.
-s	사용자의 로그인 셸 변경
-L	사용자의 계정을 잠급니다. 계정을 잠그면 사용자가 로그인할 수 없습니다.
-o(non-unique)	동일한 uid값을 갖는 계정 생성

≡ 사용자 계정 관리

≡ `userdel` 명령어

사용자 계정 정보를 삭제하는 데 사용되는 명령어입니다.

이 명령어는 사용자가 시스템에 더 이상 존재하지 않도록 하며, 관련된 파일 및 설정을 삭제할 수 있습니다.

사용 형식

```
userdel [Option] [Account_Name]
```

옵션	의미
-f 또는 --force	로그인 되어있거나, 삭제하려는 계정의 그룹을 다른 계정의 기본 그룹으로 사용하고 있을 때 강제로 삭제할 수 있습니다.
-r 또는 --remove	계정 생성시 함께 만들어진 모든 정보를 삭제해 줍니다. 사용자의 홈 디렉토리와 그 안에 포함된 모든 파일도 삭제됩니다.
-h	사용법에 대한 간단한 도움말을 제공합니다.

≡ 사용자 계정 관리

≡ `passwd` 명령어

시스템 관리자나 해당 사용자가 자신의 비밀번호를 변경할 때 사용됩니다.

사용 형식

`passwd [Option] [Account_Name]`

옵션	의미
-d	사용자의 비밀번호를 Null 값으로 지정(암호삭제)
-l	사용자의 비밀번호를 잠금 설정>Password Lock)합니다.
-u	사용자의 비밀번호를 잠금 해제>Password Unlock)합니다.
-S	계정이 잠겼는지, 비밀번호가 설정되어 있는지 등 계정에 관한 상세 정보를 알려 줍니다.
-e	사용자의 비밀번호를 즉시 만료시켜 다음 로그인시 새로운 비밀번호 입력을 유도합니다.
--stdin	사용자의 비밀번호를 표준 출력 장치로 입력 받아 생성

≡ 사용자 계정 관리

≡ `groupadd` 명령어

새로운 사용자 그룹을 생성하는 데 사용됩니다.

사용자 그룹은 시스템에서 사용자 계정을 관리할 때 중요한 역할을 하며, 권한 부여 및 파일 접근 제어 등 여러 관리 작업에 활용됩니다.

사용 형식

```
groupadd [Option] [Argument] [Group_Name]
```

사용 가능한 그룹 번호 확인

```
grep -E "GID_MIN|GID_MAX" /etc/login.defs
```

옵션	의미
-f	지정한 그룹명이 이미 존재하는 경우, 오류를 발생시키지 않고 아무 작업도 하지 않도록 합니다.
-g	그룹 생성시 GID를 명시적으로 지정합니다. 기본적으로는 시스템에서 자동으로 할당된 GID가 사용됩니다.
-o	다른 그룹과 동일한 GID를 사용하는 그룹을 생성할 수 있도록 합니다. GID는 고유해야 하지만 이 옵션을 사용하면 다른 그룹과 GID를 공유할 수 있습니다.
-r	시스템 그룹을 생성할 때 사용됩니다. 시스템 그룹은 일반적으로 시스템 관리와 관련된 사용자들을 위한 그룹으로, 그룹 ID(GID)가 100 이하로 설정됩니다.
-h	-도움말을 보여 줍니다.
-G	-그룹에 유저를 추가합니다.

≡ `groupmod` 명령어

기존 그룹을 수정하는 데 사용되는 명령어입니다.

그룹 이름을 변경하거나 그룹의 속성을 수정할 수 있습니다.

사용 형식

```
groupmod [Option] [Argument] [Group_Name]
```

옵션	의미	예시
-g	그룹의 GID(Group ID)를 변경합니다. GID는 시스템에서 그룹을 구별하기 위한 고유한 숫자입니다.	<code>sudo groupmod -g 1001 groupname</code>
-n	기존 그룹의 이름을 변경합니다.	<code>sudo groupmod -n newgroup oldgroup</code>
-o	이미 다른 그룹에 사용되고 있는 GID를 사용할 수 있게 합니다. 기본적으로 GID는 고유해야 하지만 이 옵션을 사용하면 중복 GID를 허용합니다.	<code>sudo groupmod -g 1001 -o groupname</code>
-A	그룹에 사용자를 추가합니다.	<code>sudo groupmod -A user1,user2 groupname</code>
-R	그룹에서 사용자를 제거합니다.	<code>sudo groupmod -R user1,user2 groupname</code>
-h	groupmod 명령어에 대한 도움말을 출력합니다.	<code>groupmod --help</code>

≡ 사용자 계정 관리

≡ `groupdel` 명령어

그룹을 삭제할 때 사용됩니다.

사용 형식

groupdel [옵션] 그룹명

주요 특징

항목	설명
삭제 조건	해당 그룹이 존재해야 합니다. 그룹에 소속된 사용자가 없어야 합니다. 그룹을 삭제하려면 먼저 해당 그룹에서 모든 사용자를 제거하거나 다른 기본 그룹으로 이동해야 합니다.
영향	그룹이 /etc/group 파일에서 제거됩니다. /etc/gshadow 파일에서도 해당 그룹에 대한 정보가 삭제됩니다.
제한 사항	현재 사용 중인 그룹(예: 다른 사용자 프로세스가 그룹을 참조 중인 경우)은 삭제할 수 없습니다. 그룹 삭제는 관리자 권한(sudo)이 필요합니다.

리눅스(Linux)

기본 명령어



≡ Basic Command

≡ \$변수="값"

셸 스크립트 또는 명령줄에서 변수에 문자열 "값"을 할당하는 데 사용합니다.

어떠한 데이터를 변수에 저장하고 변수의 값을 출력하거나 다른 명령어에서 사용할 수 있습니다.

사용 형식

```
$변수="값"
```

사용 예시

예시	설명
message="즐거운 시간 되세요." echo \$message	변수 message에 문자열 "즐거운 시간 되세요."을 지정합니다. echo 구문을 사용하여 문자열을 출력합니다.
echo \$message > message.txt	message의 내용을 파일 message.txt 파일에 리다이렉션합니다.

≡ Basic Command

≡ `alias(알리아스)

긴 문장의 명령어나 복잡한 명령어 시퀀스를 단축된 별명으로 정의할 수 있게 해줍니다.

자주 사용하는 명령어를 짧은 별명으로 정의할 수 있습니다.

다만 현재 세션에만 적용이 되며, 영구적으로 적용하려면 쉘 초기화 파일에 추가해야 합니다.

alias 명령어를 잘 활용하면 작업 효율을 크게 높일 수 있으며, 반복적인 명령어 입력을 줄여줍니다.

사용 구문

```
alias [ new_name ] [ =value ]
```

사용 예시

예시	설명
alias	현재 설정된 별칭 목록을 보여줍니다.
alias head3='head -n 3 java.txt'	'head3'이라는 별칭은 java.txt 파일의 앞 3줄을 읽어 들이는 역할을 합니다.
alias tail5='tail -n 5 java.txt'	'tail5'이라는 별칭은 java.txt 파일의 뒤 5줄을 읽어 들이는 역할을 합니다.
alias shut='shutdown -h now'	명령어와 옵션 지정하여 사용하기

≡ Basic Command

≡ `unalias

alias 명령으로 지정한 명령어를 해제 할때 사용하는 명령어입니다.

사용자가 이전에 설정했던 별칭을 만들었지만, 더 이상 필요하지 않을 때 사용됩니다.

사용 구문

```
unalias [옵션] [별칭]
```

사용 옵션

항목	설명
-a	모든 별칭을 제거합니다.
-v	제거된 각 별칭을 보여줍니다.

사용 예시

```
$ unalias shut # shut으로 지정했던 명령어를 해제합니다.
```

≡ Basic Command

≡ `head` 명령어

파일의 내용 중 처음부터 아래로 10줄 정보를 출력합니다.

`head` 명령어는 파일의 앞부분을 터미널에 출력하는 데 사용됩니다.

주로 파일의 첫 부분을 확인하고 싶을 때 사용됩니다.

사용 형식

head [옵션] [파일명]

사용 옵션

옵션	설명
- [num]	출력하기를 원하는 행의 수를 입력합니다. 기본 값으로 10을 사용하여 10개의 라인을 출력합니다.
-c --bytes=SIZE	출력을 원하는 용량을 정할 수 있습니다. SIZE는 b(block), k(Kilo bytes), m(Mega bytes)을 숫자뒤에 붙여 용량을 구분하여야 합니다.
-q --quiet	출력 할 때 파일명을 보여주지 않습니다.
-v --verbose	출력하는 파일명을 보여 줍니다.

사용 예시

예시	설명
head -n 5 java.txt	`java.txt` 파일의 첫 5줄을 출력합니다.

≡ Basic Command

≡ `tail` 명령어

파일의 내용 중 마지막부터 위로 10줄 정보를 출력합니다.

‘tail’ 명령어는 파일의 뒷부분을 터미널에 출력하는 데 사용됩니다.

주로 파일의 뒤쪽 부분을 확인하고 싶을 때 사용됩니다.

사용 형식

```
tail [옵션] [파일명]
```

사용 옵션

항목	설명
-n N	파일의 마지막 N줄을 출력합니다. N을 지정하지 않으면 기본 값인 10줄이 출력됩니다.

사용 예시

예시	설명
tail -n 5 java.txt	‘java.txt’ 파일의 마지막 5줄을 출력합니다.

≡ Basic Command

≡ `whoami` 명령어

현재 로그인한 사용자의 이름(UID)을 출력하는 리눅스 명령어입니다.

whoami 명령어는 현재 계정이 누구인지 확인하고자 할 때 유용합니다.

사용 구문

whoami

비슷한 명령어

항목	설명
whoami	현재 사용자의 이름 출력
who	현재 로그인된 모든 사용자 출력
id -u -n	whoami와 동일한 결과 출력
id	현재 사용자의 UID, GID, 그룹 정보 출력

≡ Basic Command

≡ `echo` 명령어

터미널에서 문자열을 출력하는 데 사용되는 기본적인 명령어입니다.

사용 구문

echo [옵션] [출력할 문자열]

주요 기능

문자열 출력
변수 출력
파일에 출력(작성하기)

주요 옵션

항목	설명	예시
-n	출력 후 기본적으로 추가되는 줄 바꿈(newline)을 제거합니다.	echo -n "Hello, World!"
-e	특정 이스케이프 시퀀스를 사용할 때 필요합니다.	echo -e "Line1\nLine2" # 줄 바꿈 echo -e "Col1\tCol2" # 탭
--help	`echo` 명령어에 대한 간단한 사용법을 확인합니다.	echo --help
--version	버전 정보 출력(GNU echo에서 지원)	echo --version

사용 예시

설명	예시
echo "Hello, World!" > file.txt	"Hello, World!"를 file.txt 파일에 기록합니다.
echo "여러분~~안녕하세요" >> file.txt	"여러분~~안녕하세요"를 file.txt 파일의 끝에 추가합니다.

≡ Basic Command

≡ `echo` 명령어

터미널에서 문자열을 출력하는 데 사용되는 기본적인 명령어입니다.

주요 옵션

항목	설명	예시
-n	출력 후 기본적으로 추가되는 줄 바꿈(newline)을 제거합니다.	echo -n "Hello, World!"
-e	특정 이스케이프 시퀀스를 사용할 때 필요합니다.	echo -e "Line1\nLine2" # 줄 바꿈 echo -e "Col1\tCol2" # 탭
--help	`echo` 명령어에 대한 간단한 사용법을 확인합니다.	echo --help
--version	버전 정보 출력(GNU echo에서 지원)	echo --version

사용 예시

설명	예시
echo "Hello, World!" > file.txt	"Hello, World!"를 file.txt 파일에 기록합니다.
echo "여러분~~안녕하세요" >> file.txt	"여러분~~안녕하세요"를 file.txt 파일의 끝에 추가합니다.

≡ Basic Command

≡ `tar` 명령어

리눅스에서 자주 사용되는 파일 묶음 및 압축을 위한 명령어 입니다.

이 명령어는 주로 백업, 파일 묶음 생성, 파일 전송 등을 위해 사용됩니다.

`tar`은 "tape archive"의 약자로, 원래 테이프 장치에 데이터를 아카이브하기 위해 개발되었습니다.

‘아카이브(archive)’는 데이터를 저장하거나 보관하는 것을 의미합니다.

특히, 데이터나 파일을 하나의 묶음으로 만들어 효율적으로 관리하거나 보관할 수 있도록 하는 과정을 말합니다.

사용 구문

```
tar [옵션] [아카이브 파일 이름] [대상 파일/디렉토리]
```

주요 옵션

항목	옵션	설명	예시
생성(Create)	-c	새 아카이브 파일 생성	tar -cvf archive.tar file1 file2
압축 해제(Extract)	-x	아카이브 파일의 내용 추출	tar -xvf archive.tar
내용 보기(List)	-t	아카이브 파일의 내용 목록 표시	tar -tvf archive.tar
압축(Compression)	-z, -j, -J	각각 gzip, bzip2, xz으로 압축하거나 해제	
기타 옵션	-v	진행 상황 표시(verbose)	
기타 옵션	-f	아카이브 파일 이름 지정	

리눅스(Linux)

중급 명령어



≡ `sudo` 명령어

일반 사용자가 **관리자 권한(root)**을 **일시적으로 사용할 수 있도록** 하는 명령어입니다.

"superuser do"의 약자로, 일반적으로 시스템 관리자 또는 root 권한으로 실행해야 하는 명령을 일시적으로 실행할 수 있게 합니다.

sudo를 사용하면 권한 부여를 통해 보안을 강화할 수 있습니다.

보안 및 시스템 안정성을 유지하면서도 필요한 경우 관리자 작업을 수행할 수 있는 방법을 제공합니다.

사용 형식

```
sudo [옵션] 명령어 [매개변수]
```

사용 옵션

항목	설명
-u 또는 --user=user_name	특정 사용자의 권한으로 명령을 실행합니다.
-g 또는 --group=group_name	특정 그룹의 권한으로 명령을 실행합니다.
-l 또는 --list	현재 사용자에게 부여된 sudo 권한을 나열합니다.

사용 예시

항목	설명	예시
패키지 설치	관리자 권한으로 패키지를 설치.	sudo apt-get install package-name
파일 접근 권한 변경	특정 파일의 권한 변경.	sudo chmod 644 /path/to/file
시스템 설정 파일 편집	시스템 호스트 이름 설정 파일 편집.	sudo nano /etc/hostname
재부팅	시스템을 재부팅.	sudo reboot

`unzip` 명령어

`zip` 형식의 압축 파일을 해제하는 데 사용됩니다.

압축된 파일을 특정 디렉토리로 풀거나, 파일 내용을 확인하거나, 일부 파일만 선택적으로 해제하는 등의 작업을 수행할 수 있습니다.

`unzip`은 `zip` 파일만 지원합니다.

다른 압축 파일 형식(예: `tar.gz`, `7z`)은 `tar`, `gzip`, `7z` 같은 다른 명령어를 사용해야 합니다.

사용 구문

```
unzip [옵션] <압축파일> [-d <디렉토리>]
```

설치 방법

대부분의 리눅스 배포판에서는 기본적으로 설치되어 있지만, 설치되어 있지 않은 경우 아래와 같이 설치할 수 있습니다.

항목	설명
Debian/Ubuntu 계열	<code>sudo apt update && sudo apt install unzip</code>
Red Hat/CentOS 계열	<code>sudo yum install unzip</code>

`unzip` 명령어

`.zip` 형식의 압축 파일을 해제하는 데 사용됩니다.

주요 옵션

항목	설명	예시
기본 사용법	`file.zip`의 내용을 현재 디렉토리로 해제합니다.	<code>unzip file.zip</code>
-d(디렉토리 지정)	`file.zip`을 지정한 `/path/to/directory`로 해제합니다.	<code>unzip file.zip -d /path/to/directory</code>
-l(압축 파일 내용 보기)	`file.zip` 안에 포함된 파일 목록과 크기를 확인합니다. 실제로 압축 해제는 하지 않습니다.	<code>unzip -l file.zip</code>
-t(압축 파일 테스트)	`file.zip`이 손상되었는지 테스트합니다.	<code>unzip -t file.zip</code>
-x(제외할 파일 지정)	`file.zip`에서 `.txt` 파일을 제외하고 압축을 해제합니다.	<code>unzip file.zip -x "*.txt"</code>
-j(디렉토리 구조 무시)	압축 해제 시 디렉토리 구조를 무시하고 모든 파일을 현재 디렉토리에 풀어놓습니다.	<code>unzip -j file.zip</code>
-n(기존 파일 덮어쓰지 않기)	현재 디렉토리에 동일한 이름의 파일이 있으면 새로 덮어쓰지 않고 건너뛵니다.	<code>unzip -n file.zip</code>
-o(기존 파일 덮어쓰기)	기존에 동일한 이름의 파일이 있어도 덮어쓰기를 수행합니다.	<code>unzip -o file.zip</code>
-q(출력 최소화)	압축 해제 시 진행 상태를 출력하지 않습니다.	<code>unzip -q file.zip</code>

≡ `unzip` 명령어

`zip` 형식의 압축 파일을 해제하는 데 사용됩니다.

용용 예제

항목	설명	예시
전체 압축 해제	"example.zip"이라는 파일을 현재 작업 디렉토리에 압축 해제합니다.	<code>unzip example.zip</code>
특정 파일만 해제하기	`file.zip`에서 `specific_file.txt`만 해제합니다.	<code>unzip file.zip specific_file.txt</code>
암호화된 압축 파일 해제	비밀 번호로 보호된 파일을 해제할 때 사용합니다.	<code>unzip -P password file.zip</code>
디렉토리 구조를 유지하면서 특정 디렉토리로 해제	압축 파일의 디렉토리 구조를 유지하며 `/home/user/extracted`에 풀어냅니다.	<code>unzip example.zip -d /home/user/extracted</code>

`tree` 명령어

디렉토리 구조를 트리 형태로 시각적으로 보여주는 유틸리티입니다.

파일과 디렉토리를 계층적으로 볼 수 있어 디렉토리 구조를 이해하는 데 유용합니다.

tree는 일반적으로 기본적으로 설치되어 있지 않지만, 대부분의 리눅스 배포판에서 쉽게 설치할 수 있습니다.

설치하기

```
sudo apt install -y tree
```

사용 형식

```
tree [옵션] [디렉토리 경로]
```

사용 옵션

항목	설명
-a 또는 --all	숨겨진 파일 및 디렉토리를 포함하여 모든 항목을 표시합니다.
-d 또는 --dirs-only	파일이 아닌 디렉토리만 표시합니다.
-L level 또는 --max-depth=level	표시할 최대 깊이를 지정합니다.
-f 또는 --fullpath	전체 경로를 표시합니다.
-i 또는 --charset	출력에서 사용할 문자 세트를 지정합니다.
-h 또는 --help	도움말 메시지를 표시합니다.

`tree` 명령어

디렉토리 구조를 트리 형태로 시각적으로 보여주는 유틸리티입니다.

사용 예시

항목	예시	설명
특정 디렉토리 지정	<code>tree /path/to/directory</code>	지정된 디렉토리의 구조를 트리 형태로 출력합니다.
깊이 제한	<code>tree -L level</code>	트리 구조의 출력 깊이를 제한합니다. 예를 들어, <code>-L 2</code> 는 2단계 깊이까지만 출력합니다.
모든 파일과 디렉토리 표시	<code>tree -a</code>	숨김 파일과 디렉토리를 포함하여 모든 파일과 디렉토리를 표시합니다.
파일 크기 표시	<code>tree -s</code>	각 파일의 크기를 표시합니다.
파일 타입 표시	<code>tree -F</code>	파일 이름 끝에 파일 타입을 나타내는 문자를 추가합니다.(예: <code>/</code> 는 디렉토리, <code>*</code> 는 실행 파일)
출력 파일 저장	<code>tree -o filename.txt</code>	트리 구조 출력을 지정한 파일에 저장합니다.
깊이별 색상 구분	<code>tree -C</code>	색상을 사용하여 출력 결과를 구분합니다.
특정 파일 패턴 매칭	<code>tree -P pattern</code>	지정한 패턴과 일치하는 파일만 출력합니다. 예를 들어, <code>-P '*.txt'</code> 는 <code>.txt</code> 파일만 출력합니다.
디렉토리 크기 표시	<code>tree -h</code>	각 파일과 디렉토리의 크기를 인간이 읽을 수 있는 형식(예: KB, MB)으로 표시합니다.

`wget` 명령어

Linux/Unix 및 Windows 환경에서 **웹에서 파일을 다운로드**하는 데 사용됩니다.

HTTP, HTTPS, FTP 같은 프로토콜을 지원하며, 배치 다운로드와 같은 자동화된 작업에 유용합니다.

주요 특징

- 비동기 다운로드 : 사용자가 입력을 기다리지 않고 다운로드를 계속합니다.
- 중단된 다운로드 재개 : 네트워크가 끊기더라도 파일을 이어 받을 수 있습니다.
- 배치 다운로드 : 여러 URL을 한 번에 다운로드할 수 있습니다.
- 웹 크롤링 : 웹 페이지 전체를 다운로드하여 로컬에 저장할 수 있습니다.

사용 구문

wget [옵션] [URL]

사용 옵션

항목	예시	설명
파일 다운로드	wget https://example.com/file.zip	지정된 URL에서 `file.zip` 파일을 다운로드합니다.
파일 이름 지정	wget -O custom_name.zip https://example.com/file.zip	다운로드한 파일을 `custom_name.zip`으로 저장합니다.
중단된 다운로드 재개	wget -c https://example.com/largefile.zip	다운로드 중 중단된 파일을 이어서 받습니다.

≡ `ssh` 명령어

네트워크를 통해 원격 서버에 안전하게 접속하여 명령을 실행하기 위한 프로토콜 및 명령어입니다.

사용 구문

```
ssh [옵션] 사용자@서버주소
```

특징

모든 통신 내용 암호화
비밀번호 또는 키 기반 인증
Telnet, FTP 같은 평문 통신 방식의 대체

사용 옵션

옵션	설명
-p	SSH 포트 지정(기본 22)
-i	개인 키 파일 지정
-v	접속 과정 디버깅 출력
-X	X11 포워딩
-N	명령 실행 없이 터널링
-L	로컬 포트 포워딩
-R	원격 포트 포워딩

주요 파일

옵션	설명
~/.ssh/id_rsa	개인 키
~/.ssh/id_rsa.pub	공개 키
~/.ssh/authorized_keys	접속 허용 키 목록
/etc/ssh/sshd_config	서버 설정

≡ `scp` 명령어

SSH 프로토콜을 이용하여 원격 서버로 파일/디렉토리를 복사하는 명령어입니다.

주요 특징

데이터 전송 구간 암호화

SSH 계정 인증 사용

별도의 서버 설정 없이 사용 가능(SSH만 있으면 됨)

사용 예시

```
scp -r ~/git_sample_01/WebServer01 panda@192.168.0.110:/home/panda/
```

파일 하나만 복사

```
scp file.txt panda@192.168.0.110:/home/panda/
```

원격 → 로컬 복사

```
scp -r panda@192.168.0.110:/home/panda/WebServer01 ~/
```

포트 지정

```
scp -P 2222 file.txt panda@192.168.0.110:/home/panda/
```

`curl` 명령어

URL을 사용하여 데이터를 전송하거나 가져 오는 데 사용하는 명령줄 도구 및 라이브러리입니다.

HTTP, HTTPS, FTP, SCP, SFTP, SMTP, IMAP 등 다양한 프로토콜을 지원하며, 이를 통해 웹 서버와 통신합니다.

curl은 명령줄에서 데이터를 전송하거나 가져 오는 데 유용하며, Scripting과 자동화에 자주 사용됩니다.

사용 형식

```
curl [옵션] [URL]
```

사용 옵션

항목	설명
-X, --request <command>	HTTP 요청 메서드를 지정합니다. GET, POST, PUT 등을 지정할 수 있습니다.
-d, --data <data>	POST 요청의 데이터를 지정합니다.
-H, --header <header>	HTTP 요청 헤더를 지정합니다.
-i, --include	서버 응답에 헤더를 포함하여 출력합니다.
-o, --output <file>	출력을 파일로 저장합니다.
-O, --remote-name	서버로부터 받은 파일을 현재 디렉토리에 원격 파일 이름으로 저장합니다.
-L, --location	URL이 리다이렉션될 경우 이를 따라갑니다.
-u, --user <user:password>	HTTP 기본 인증을 위해 인증 정보 전달(사용자 이름과 비밀번호)을 지정합니다.
-v, --verbose	자세한 정보를 출력합니다.

[다음 페이지에 계속...](#)

≡ `curl` 명령어

URL을 사용하여 데이터를 전송하거나 가져 오는 데 사용하는 명령줄 도구 및 라이브러리입니다.

사용 옵션

항목	설명
-f	실패 시 오류 메시지를 출력하지 않고 조용히 실패합니다.
-s	비활성 모드(silent)로 동작하여 진행 상태를 표시하지 않습니다.
-S	오류 발생 시, 오류 메시지를 출력합니다.(-s 옵션과 조합해 사용)

`curl` 명령어

URL을 사용하여 데이터를 전송하거나 가져 오는 명령줄 도구 및 라이브러리입니다.

사용 예시

```
for i in {1..20}; do curl $ALB --silent; done | sort | uniq -c | sort -nr
```

for i in {1..20}: 1부터 20까지의 숫자에 대해 루프를 실행합니다.

curl \$ALB --silent: curl 명령어를 사용하여 ALB에 HTTP 요청을 보냅니다.

--silent 옵션은 진행 상태나 오류 메시지를 출력하지 않게 합니다.

done: 루프의 끝을 나타냅니다.

|: 파이프(|)는 이전 명령의 출력을 다음 명령의 입력으로 전달합니다.

sort: 표준 입력을 정렬합니다.

uniq -c: 중복된 라인을 제거하고, 각 라인이 몇 번 나타났는지를 세어 표시합니다.

sort -nr: 숫자 기준으로 내림차순 정렬합니다.

`curl` 명령어

URL을 사용하여 데이터를 전송하거나 가져 오는 명령줄 도구 및 라이브러리입니다.

사용 예시

예시	설명
<code>curl https://www.example.com</code>	"https://www.example.com"에서 웹 페이지를 읽어 옵니다.
<code>curl -X POST -d "username=myuser&password=mypassword" https://www.example.com/login</code>	"https://www.example.com/login"에 사용자 이름과 비밀번호를 전송하는 POST 요청을 보냅니다.
<code>curl -H "Accept: application/json" http://api.example.com/data</code>	GET 요청으로 JSON 데이터 가져오기
<code>curl -X POST -F "name=John" -F "age=30" http://example.com/submit</code>	POST 요청으로 폼 데이터 보내기
<code>curl -X PUT -H "Content-Type: application/json" -d '{"name":"John", "age":30}' http://example.com/update</code>	헤더와 데이터를 포함한 PUT 요청 보내기
<code>curl -o newname.jpg http://example.com/image.jpg</code>	다운로드 파일을 다른 이름으로 저장
<code>curl -v http://example.com > output.log 2>&1</code>	HTTP 요청 로그를 파일에 저장

SNMP(Simple Network Management Protocol) 명령어

네트워크 장비들(라우터, 스위치, 서버 등)을 모니터링하고 관리하는 프로토콜입니다.

참조 사이트) <https://www.site24x7.com/network/what-is-snmp.html>

SNMP 관리자와 Agent간에 메시지를 주고 받는 방식으로 동작합니다.

네트워크 장비들의 성능 모니터링, 구성 변경, 장애 진단 등 사용한 용도로 사용됩니다.

snmpget 명령어는 SNMP(Simple Network Management Protocol)를 사용하여 네트워크 장치에서 특정 정보를 가져 오는 데 사용됩니다.

사용 구문

```
snmpget [OPTIONS] AGENT OID
```

기본 OID 정보

옵션	설명
1.3.6.1.2.1.1.5.0	장치의 시스템 이름
1.3.6.1.2.1.1.3.0	시스템 가동 시간(sysUpTime)

항목	설명
Agent	네트워크 장비들에 설치되는 항목입니다. SNMP 에이전트의 호스트 이름 또는 IP 주소 등등
MIB	Management Information Base 네트워크 장비들의 상태 정보를 저장하고 있는 데이터 베이스입니다.
OID	요청할 MIB 객체의 식별자입니다.

SNMP(Simple Network Management Protocol) 명령어

네트워크 장비들(라우터, 스위치, 서버 등)을 모니터링하고 관리하는 프로토콜입니다.

관리자는 네트워크 디바이스의 상태를 실시간으로 확인하고 문제를 진단할 수 있습니다.

사용 형식

AGENT는 SNMP 에이전트의 호스트 이름 또는 IP 주소이며, OID는 요청할 MIB 객체의 식별자입니다.

```
snmpget [OPTIONS] AGENT OID
```

```
snmpwalk [OPTIONS] AGENT [OID]
```

```
snmpset [OPTIONS] AGENT OID TYPE VALUE
```

사용 예시

```
# snmpget [OPTIONS] AGENT OID
```

```
snmpget -v2c -c public $EC21 1.3.6.1.2.1.1.5.0
```

```
SNMPv2-MIB::sysName.0 = STRING: SERVER1
```

"SNMPv2-MIB::sysName.0"는 MIB 객체의 이름이고, "STRING: SERVER1"는 MIB 객체의 값입니다.

```
snmpget -v2c -c public $EC21 1.3.6.1.2.1.1.3.0
```

```
DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks:(398077) 1:06:20.77
```

SNMP(Simple Network Management Protocol) 명령어

네트워크 장비들(라우터, 스위치, 서버 등)을 모니터링하고 관리하는 프로토콜입니다.

사용 옵션

항목	설명
-v	SNMP 버전을 지정합니다. 주로 '1', '2c', '3'이 사용됩니다.
-c	커뮤니티 문자열을 지정합니다. SNMPv1 및 SNMPv2c에서는 커뮤니티 문자열이 필요합니다.
-u	SNMPv3에서 사용자 이름을 지정합니다.
-l	보안 수준을 지정합니다. 보통은 auth, noAuthNoPriv, authPriv 등이 사용됩니다.
-a	인증 프로토콜을 지정합니다. 주로 'MD5'나 'SHA'가 사용됩니다.
-x	암호화 프로토콜을 지정합니다. 주로 'DES', 'AES' 등이 사용됩니다.
-A	인증 암호를 지정합니다.
-X	암호화 암호를 지정합니다.

`dig(Domain Information Groper) 명령어

도메인 이름 시스템(DNS) 관련 쿼리를 수행하는 데 사용되는 명령줄 도구입니다.

네트워크 관리자 및 시스템 관리자가 DNS 문제를 진단하고 해결하기 위해 사용됩니다.

dig 명령어는 DNS 조회를 통해 도메인 이름에 대한 다양한 정보를 얻을 수 있습니다.

사용 구문

```
dig [옵션] [도메인 이름] [TYPE]
```

사용 옵션

옵션	설명
@server	특정 DNS 서버에 질의합니다.
-x	역방향 DNS 조회를 수행합니다.
+trace	DNS 조회 경로(쿼리의 경로)를 따라 DNS 쿼리를 추적합니다.
+short	간략한 출력 형식으로 결과를 보여줍니다.
+nocmd	명령어 부분을 표시하지 않습니다.
+nocomments	주석을 표시하지 않습니다.
+noquestion	질의를 표시하지 않습니다.
+noauthority	권한 섹션을 표시하지 않습니다.
+noadditional	추가 섹션을 표시하지 않습니다.

dig(Domain Information Groper) 명령어

도메인 이름 시스템(DNS) 관련 쿼리를 수행하는 데 사용되는 명령줄 도구입니다.

사용 예시

예시	설명
dig example.com	도메인에 대한 기본 조회(Default Query for a Domain)
dig example.com @8.8.8.8	특정 DNS 서버에 질의(Query to a Specific DNS Server)
dig example.com MX	특정 유형의 레코드 조회(Query for a Specific Type of Record)
dig -x 192.0.2.1	역방향 DNS 조회(Reverse DNS Lookup)
dig example.com +trace	추적(Trace)
dig ap-northeast-2.elb.amazonaws.com +short	간략한 결과 표시(Show Short Results)

ALB의 DNS 이름을 지정합니다.

ALB=ALB-1599450422.ap-northeast-2.elb.amazonaws.com

2개의 유동 공유 IP가 출력됩니다.

dig \$ALB +short

≡ `growpart` 명령어

파티션의 크기를 동적으로 조정하고자 할 때 사용되는 명령어입니다.

cloud-utils 패키지에 포함되어 있으며, 주로 클라우드 환경에서 디스크 크기를 확장한 후, 해당 디스크의 파티션 크기를 조정할 때 유용하게 사용됩니다.
growpart는 GNU parted 프로젝트의 일부로 제공되며, 클라우드 환경에서 디스크 확장이 필요한 경우 자주 사용됩니다.

필수 패키지 설치

growpart는 cloud-utils 패키지에 포함되어 있으므로, 설치가 필요합니다.

```
sudo apt install cloud-guest-utils # Ubuntu/Debian  
sudo yum install cloud-utils-growpart # CentOS/RHEL
```

사용 구문

```
growpart [options] DISK PARTITION
```

사용 옵션

항목	설명
-h, --help	도움말 메시지를 표시합니다.
-N, --dry-run	실제로 변경하지 않고 어떻게 동작할지 시뮬레이션합니다.
-v, --verbose	자세한 정보를 출력합니다.
-q, --quiet	조용한 모드, 최소한의 출력만 표시합니다.

[다음 페이지에 계속](#)

`growpart` 명령어

파티션의 크기를 동적으로 조정하고자 할 때 사용되는 명령어입니다.

cloud-utils 패키지에 포함되어 있으며, 주로 클라우드 환경에서 디스크 크기를 확장한 후, 해당 디스크의 파티션 크기를 조정할 때 유용하게 사용됩니다. growpart는 GNU parted 프로젝트의 일부로 제공되며, 클라우드 환경에서 디스크 확장이 필요한 경우 자주 사용됩니다.

사용 예시

예시	설명
<code>sudo growpart /dev/sda 1</code>	디스크 /dev/sda의 첫 번째 파티션 크기 확장 이 명령은 /dev/sda1 파티션의 크기를 가능한 한 최대로 확장합니다.
<code>sudo growpart --dry-run /dev/sda 1</code>	실제로 변경하지 않고 시뮬레이션 어떻게 변경이 될지 시뮬레이션만 하며, 실제로 파티션을 변경하지는 않습니다.
<code>sudo growpart --verbose /dev/sda 1</code>	자세한 정보와 함께 파티션 크기 확장 이 명령은 확장 과정을 자세히 출력합니다.
<code>sudo growpart --quiet /dev/sda 1</code>	최소한의 출력으로 파티션 크기 확장 이 명령은 성공 또는 오류 메시지 외에는 아무것도 출력하지 않습니다.

≡ `xfs\_growfs` 명령어

XFS 파일 시스템의 크기를 온라인으로 확장할 수 있는 유틸리티입니다.

파일 시스템이 마운트된 상태에서도 파티션이나 볼륨의 크기를 확장할 수 있어 매우 유용합니다.

이 명령어를 사용하면 기존 XFS 파일 시스템에 추가한 새 공간을 인식하고 사용할 수 있도록 확장할 수 있습니다.

사용 구문

[마운트 지점] : 크기를 확장할 파일 시스템이 마운트된 디렉토리.

xfs_growfs [옵션] [마운트 지점]

사용 옵션

항목	설명
-d, --data	데이터 영역을 최대 크기로 확장합니다.
-L, --log	로그 영역을 최대 크기로 확장합니다.
-r, --rt	실시간 영역을 최대 크기로 확장합니다.
-n, --dry-run	실제로 변경하지 않고, 어떻게 동작할지 시뮬레이션합니다.
-V, --version	버전 정보를 출력합니다.

≡ `xfs\_growfs` 명령어

XFS 파일 시스템의 크기를 온라인으로 확장할 수 있는 유틸리티입니다.

사용 예시

예시	설명
<code>sudo xfs_growfs /mnt</code>	XFS 파일 시스템을 최대 크기로 확장 여기서 /mnt는 XFS 파일 시스템이 마운트된 디렉토리입니다.
<code>sudo xfs_growfs -d /mnt</code>	데이터 영역을 최대 크기로 확장 이 명령은 데이터 영역만 확장합니다.
<code>sudo xfs_growfs -L /mnt</code>	로그 영역을 최대 크기로 확장 이 명령은 로그 영역을 확장합니다.
<code>sudo xfs_growfs -r /mnt</code>	실시간 영역을 최대 크기로 확장 이 명령은 실시간 영역을 확장합니다.
<code>sudo xfs_growfs -n /mnt</code>	실제로 변경하지 않고 시뮬레이션 이 명령은 변경 사항을 적용하지 않고, 어떤 변경이 일어날지 시뮬레이션만 합니다.

fallocate 명령어

파일을 생성하거나 기존 파일의 크기를 조정하는 데 사용됩니다.

파일 시스템의 효율성을 높이기 위해 파일의 공간을 미리 할당할 수 있게 합니다.

지정된 크기의 파일을 빠르게 생성하거나 크기를 변경할 수 있습니다.

fallocate는 특히 XFS, ext4 등의 파일 시스템에서 효율적으로 작동합니다.

특히 큰 파일을 생성하거나 파일 내의 공간을 관리해야 하는 상황에서 유용하게 사용될 수 있습니다.

사용 구문

```
fallocate [옵션] <파일명> -l <크기>
```

사용 옵션

옵션	설명
-l, --length <length>	파일의 크기를 지정합니다. <length>는 숫자와 단위(k, m, g 등)를 사용하여 표현합니다.
-o, --offset <offset>	작업을 시작할 오프셋을 지정합니다.
-n, --keep-size	파일 크기를 변경하지 않습니다.
-p, --punch-hole	지정된 오프셋과 길이로 파일에서 할당된 공간을 해제합니다(파일의 회소화).
-z, --zero-range	지정된 범위의 내용을 0으로 채웁니다.

≡ `fallocate` 명령어

파일을 생성하거나 기존 파일의 크기를 조정하는 데 사용됩니다.

사용 예시

예시	설명
<code>fallocate -l 100M myfile</code>	크기가 100MB인 파일 생성 myfile이라는 파일을 생성하고, 크기를 100MB로 설정합니다.
<code>fallocate -l 1G existingfile</code>	기존 파일의 크기를 1GB로 설정: existingfile이라는 파일의 크기를 1GB로 설정합니다.
<code>fallocate -p -o 1M -l 2M myfile</code>	파일의 특정 범위에서 할당된 공간 해제 myfile의 1MB 오프셋에서 시작하여 2MB의 할당된 공간을 해제합니다.
<code>fallocate --collapse-range -o 1M -l 2M myfile</code>	파일의 특정 범위를 제거하고 후속 데이터를 앞으로 당기기 myfile의 1MB 오프셋에서 시작하여 2MB의 범위를 제거하고 후속 데이터를 앞으로 당깁니다.
<code>fallocate -z -o 1M -l 2M myfile</code>	파일의 특정 범위를 0으로 채우기 myfile의 1MB 오프셋에서 시작하여 2MB의 범위를 0으로 채웁니다.
<code>fallocate --insert-range -o 1M -l 2M myfile</code>	파일의 특정 위치에 범위를 삽입 myfile의 1MB 오프셋에서 시작하여 2MB의 범위를 삽입합니다.

yum(Yellowdog Updater, Modified)

패키지를 설치, 업데이트, 삭제, 검색하는 데 사용되는 강력한 명령줄 패키지 관리자입니다.

'Yellowdog Updater, Modified'의 약어이고, CentOS, Red Hat Linux, Fedora 등의 RPM 기반 리눅스 배포판에서 패키지 관리에 사용됩니다. CentOS, RHEL(Red Hat Enterprise Linux), Fedora 등의 RPM 기반 리눅스 배포판에서 패키지 관리에 사용됩니다. install, update, remove, search 등의 명령어 사용이 가능합니다.

사용 형식

```
yum [OPTIONS] [COMMAND] [PACKAGE...]
```

사용 옵션

항목	설명
-y, --assumeyes	패키지 설치 또는 제거 시 사용자 확인을 건너뛰고 자동으로 yes로 간주합니다.
-q, --quiet	출력을 최소화하여 조용한 모드로 작동합니다.
-v, --verbose	자세한 출력을 표시합니다.
--enablerepo=REPOID	특정 저장소를 활성화합니다.
--disablerepo=REPOID	특정 저장소를 비활성화합니다.
--nogpgcheck	GPG 키 확인을 건너뛰고 패키지를 설치합니다.
--exclude=PACKAGE	특정 패키지를 설치에서 제외합니다.
--downloadonly	패키지를 다운로드하지만 설치하지 않습니다.

[다음 페이지에 계속...](#)

yum(Yellowdog Updater, Modified)

패키지를 설치, 업데이트, 삭제, 검색하는 데 사용되는 강력한 명령줄 패키지 관리자입니다.

사용 예시

예시	설명
yum search <keyword>	패키지 검색(Search Packages)
yum info <package_name>	패키지 정보 표시(Show Package Information)
yum install <package_name>	패키지 설치(Install Package)
yum remove <package_name>	패키지 제거(Remove Package)
yum update	시스템 업그레이드(Upgrade System)
yum update <package_name>	특정 패키지 업그레이드(Upgrade Specific Package)
yum install <package_name> --resolve	의존성 해결(Resolve Dependencies)
yum clean all	캐시 지우기(Clean Cache)

```
yum install httpd -y
```

`apt(Advanced Package Tool)

Debian 계열의 Linux 배포판에서 **소프트웨어 패키지를 관리**하기 위한 명령줄 도구입니다.

패키지

패키지	용도	패키지	용도
apache2	웹 서버 Apache(아파치)	tree	디렉토리 구조 확인
curl	API 테스트, 서버 상태 확인	ufw	우분투 방화벽
git	소스 관리	unzip	압축 해제
gradle	그레이들	vim	설정 파일 편집
htop	프로세스/메모리 모니터링	wget	파일 다운로드
maven	메이븐	zip	파일 압축하기
mysql-client	MySQL 클라이언트	<pre> java -version javac -version mysql --version git --version mvn -v </pre>	
mysql-server	MySQL 서버		
net-tools	netstat 등 네트워크 확인		
nginx	웹 서버 Nginx(엔진엑스)		
openjdk-17-jdk	Java(Spring Boot 필수)		
openssh-server	원격 접속 관련 패키지		

`apt(Advanced Package Tool)

Debian 계열의 Linux 배포판에서 **소프트웨어 패키지를 관리**하기 위한 명령줄 도구입니다.

Debian 계열의 Linux 배포판 중에서 가장 널리 알려진 것은 우분투(ubuntu)입니다.

주요 특징

항목	설명	예시
패키지 설치	사용자가 지정한 소프트웨어를 저장소(repository)에서 다운로드하고 설치합니다.	<code>sudo apt install cloud-guest-utils</code>
패키지 업데이트	시스템에 설치된 모든 패키지를 최신 버전으로 업그레이드합니다.	<code>sudo apt update</code> <code>sudo apt upgrade</code>
패키지 제거	설치된 소프트웨어를 시스템에서 삭제합니다.	<code>sudo apt remove <패키지명></code>
의존성 관리	소프트웨어를 설치하거나 제거할 때 관련된 다른 패키지(의존성)를 자동으로 처리합니다.	

≡ `systemctl` 명령어

systemd(**시스템 및 서비스 매니저**)와 상호 작용하기 위해 사용하는 명령줄 도구입니다.

Systemd 명령어는 **System Control**의 줄임 말로 Linux 시스템의 초기화 시스템으로, 부팅, 프로세스 관리, 서비스 관리 등을 담당합니다. systemctl 명령은 서비스를 시작, 중지, 다시 시작, 상태 확인 등의 작업을 수행하는 데 사용됩니다.

주요 기능

항목	설명
서비스 관리	서비스 시작, 중지, 재시작, 상태 확인 등을 수행합니다.
타이머 관리	특정 시간에 작업을 실행하도록 타이머 유닛을 설정하고 관리합니다.
부팅 프로세스 제어	시스템이 부팅될 때 시작할 유닛(서비스, 타이머 등)을 지정합니다.
런레벨(타겟) 관리	Systemd에서 사용하는 런레벨(target)을 전환하거나 설정합니다.

사용 형식

COMMAND: 수행할 작업을 지정하는 명령, NAME: 대상 서비스나 유닛의 이름.
systemctl [OPTIONS] COMMAND [NAME]

`systemctl` 명령어

systemd(시스템 및 서비스 매니저)와 상호 작용하기 위해 사용하는 명령줄 도구입니다.

사용 예시

예시	설명
<code>systemctl start <service-name></code>	서비스 시작
<code>systemctl stop <service-name></code>	서비스 중지
<code>systemctl restart <service-name></code>	서비스 다시 시작
<code>systemctl reload <service-name></code>	서비스 재로드(구성을 다시 읽음)
<code>systemctl enable <service-name></code>	서비스 활성화(부팅 시 자동 시작)
<code>systemctl disable <service-name></code>	서비스 비활성화(부팅 시 자동 시작 해제)
<code>systemctl status <service-name></code>	서비스 상태 확인
<code>systemctl reboot</code>	시스템 재부팅
<code>systemctl poweroff</code>	시스템 종료

```
systemctl start httpd
```

`httpd` 명령어

Apache HTTP Server의 실행 파일 이름으로, 웹 서버 소프트웨어를 실행하는 데 사용됩니다.

Apache Software Foundation에서 개발한 오픈 소스 웹 서버로, 인터넷에서 가장 널리 사용되는 웹 서버 중 하나입니다.

httpd란?

httpd는 HTTP Daemon(HTTP 데몬)의 약자입니다.

"데몬"은 백그라운드에서 실행되며 사용자 요청을 처리하는 프로그램을 의미합니다.

httpd는 HTTP 프로토콜을 통해 클라이언트(브라우저 등) 요청을 처리하고 응답(HTML, 이미지, 파일 등)을 반환합니다.

주요 역할

항목	설명
클라이언트 요청 처리	사용자가 웹 브라우저를 통해 URL을 요청하면 httpd는 이 요청을 처리하고 해당 콘텐츠를 반환합니다.
정적 및 동적 콘텐츠 제공	정적 콘텐츠: HTML 파일, 이미지, CSS 등 고정된 파일을 제공. 동적 콘텐츠: PHP, Python, Perl 등으로 생성된 응답 처리 지원.
가상 호스팅	하나의 서버에서 여러 도메인이나 웹사이트를 호스팅 가능.
보안 및 인증	SSL/TLS를 통한 암호화 지원. 사용자 인증 및 권한 설정.
모듈 확장	다양한 모듈을 사용하여 기능 확장 가능(예: `mod_ssl`, `mod_rewrite` 등).

`tee` 명령어

표준 입력(stdin)으로 들어오는 데이터를 표준 출력(stdout)으로 출력하는 동시에 파일로도 저장하는 명령어입니다.

일반적으로 파이프라인에서 다른 명령어의 출력을 파일로 기록하면서 동시에 화면에 출력할 때 사용됩니다.

기본적으로 파일에 데이터를 덮어쓰는 방식으로 작동하지만, -a 옵션을 사용하면 추가 모드로 작동하여 기존 내용을 덮어 쓰지 않고 새 데이터를 파일에 추가합니다.

사용 형식

```
tee [옵션] [파일명...]
```

사용 옵션

항목	설명
-a	파일에 내용을 덧붙입니다.(기존 내용 덮어쓰지 않음)
-i	대소문자 구분 없이 동작합니다.
-p	출력이 실패할 경우 종료하지 않고 계속해서 다른 출력 파일에 데이터를 기록합니다.

사용 예시

항목	설명
echo "Hello, world!" tee output.txt	출력: 터미널에 "Hello, world!"가 출력됩니다. 파일: output.txt 파일에 "Hello, world!" 내용이 저장됩니다.
echo "Line 1" tee -a output.txt echo "Line 2" tee -a output.txt	파일 output.txt에 "Line 1"이 기록되고, 이후 "Line 2"가 추가됩니다.
echo "Log entry" tee file1.txt file2.txt	"Log entry"를 file1.txt와 file2.txt 두 파일에 동시에 기록합니다.
ls -l tee directory_list.txt	ls -l 명령의 출력을 터미널에 출력하는 동시에 directory_list.txt 파일에 저장합니다.

`netstat` 명령어

네트워크의 상태 정보를 확인하는 명령어입니다.

네트워크 연결, 라우팅 테이블, 포트 등을 확인할 수 있습니다.

사용 예시

다음은 MySQL 서버가 제대로 실행 중인지, 포트 3306에서 연결을 대기하고 있는지, 외부에서 접근 가능한 상태인지를 확인하는 데 사용됩니다.

```
sudo netstat -tuln | grep 3306
```

주요 옵션

항목	설명
-t	TCP 포트만 표시
-u	UDP 포트도 함께 표시
-l	현재 리스닝 중인 포트(열려 있는 포트) 표시
-n	도메인 이름 대신 숫자(IP 주소와 포트 번호)로 출력

리눅스(Linux)

우분투 방화벽



ufw(Uncomplicated Firewall)

리눅스 서버에 "어떤 포트로 들어오는 통신을 허용할 것인가?"를 관리해주는 방화벽입니다.

기본 개념

항목	설명
포트	서비스 출입문(80, 443, 9000, 3306 등)
ufw	출입문 관리인
allow	문 열어줌
deny	문 잠금

관련 명령어

항목	설명
<code>sudo ufw status</code>	ufw 허용 규칙(상태) 확인 상태가 inactive(방화벽 꺼짐), active(규칙 적용 중)
<code>sudo ufw status grep ALLOW</code>	ALLOW만 보기
<code>sudo ufw status verbose</code>	상세 정보
<code>sudo ufw status numbered</code>	번호 포함
<code>sudo ufw allow 9000</code>	9000번 포트 열어줌
<code>sudo ufw allow ssh</code> <code>sudo ufw allow 22</code>	SSH(원격 접속 유지 필수)
<code>sudo ufw enable</code>	ufw 활성화
<code>sudo ufw allow 3306</code>	MySQL 포트 허용
<code>sudo ufw reload</code>	Firewall reloaded

리눅스(Linux)

VI Editor



≡ What is `VI editor`?

Emacs와 함께 유닉스 환경에서 많이 쓰이는 문서 편집기입니다.

vi 편집기

1976년 빌 조이가 초기 BSD 릴리즈에 포함될 편집기로 제작
줄 단위 편집기가 아닌 한 화면을 편집하는 Visual editor 이라는 뜻에서 유래
명령, 실행(EX), 입력 모드가 있습니다.

사용 예시

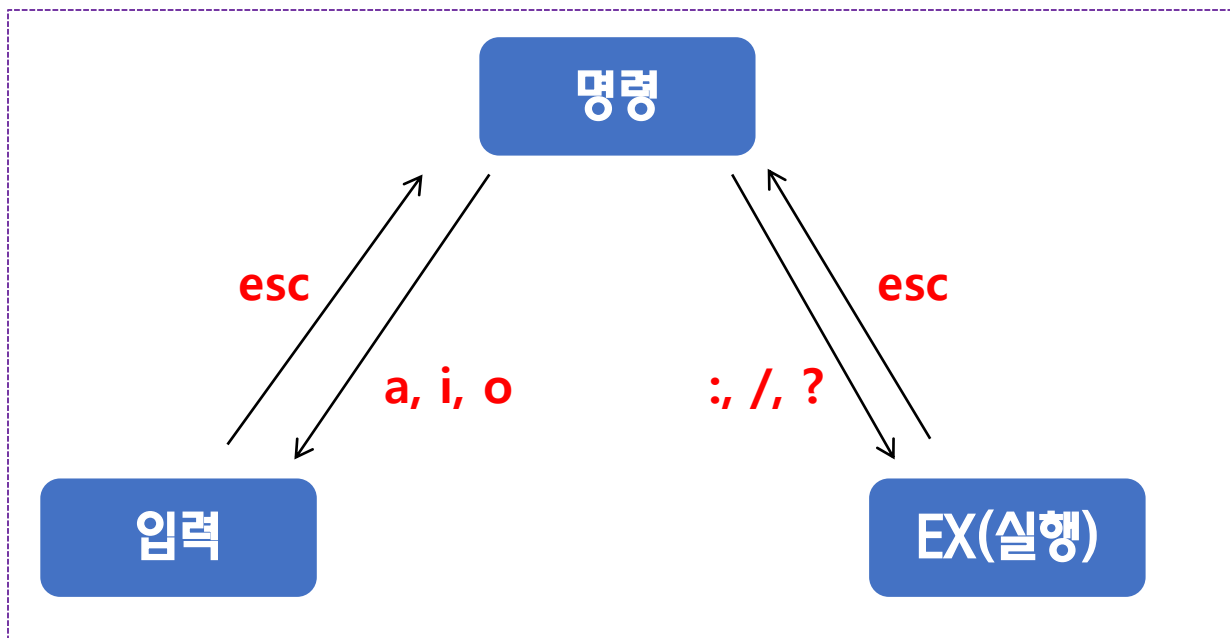
예시	설명
vi + filename	기존 파일의 맨 마지막 라인에 커서 위치
vi + 숫자 filename	기존 파일의 원하는 행부터 편집
vi + /문자열 filename	기존 파일의 원하는 문자열부터 편집

≡ VI Editor

≡ What is `VI editor?

Emacs와 함께 유닉스 환경에서 많이 쓰이는 문서 편집기입니다.

vi의 3가지 모드



모드	설명
명령 모드(기본 모드)	커서 이동, 수정, 삭제
입력 모드	명령 모드에서 글자 1개를 입력합니다.
실행 모드	검색, 종료, 명령어(저장, 읽기, 외부 명령 실행 등)

≡ 명령 모드

`명령` 모드는 Vi의 기본 모드입니다.

명령 모드

vi 편집기가 최초 실행이 되는 경우
입력 모드에서 ESC 키를 누른 경우
커서 이동, 문자열 수정, Copy & Paste 등의 작업이 필요한 경우
엔터키를 누를 필요 없이 각각의 명령에 따른 버튼을 누르면 바로 실행이 됩니다.

참고로 다음 단축 키들은 키보드 자판에 연속으로 배치되어 있습니다.

단축키	설명
j	커서를 아래로 이동
k	커서를 위로 이동
h	커서를 왼쪽으로 이동
l(엘)	커서를 오른쪽으로 이동

≡ 명령 모드

기본적인 커서 이동에 대한 명령어 목록입니다.

명령어	설명
w	한 단어씩 우측으로 이동하여 다음 단어의 첫 글자에 커서가 놓입니다.
b	한 단어씩 좌측으로 이동하여 단어의 첫 글자에 커서가 놓입니다.
W	기본적으로 w와 동일한 기능을 수행하되, 특수 문자는 배제합니다.
B	기본적으로 b와 동일한 기능을 수행하되, 특수 문자는 배제합니다.
e	한 단어씩 우측으로 이동하여 다음 단어의 끝 글자에 커서가 놓입니다.
0(zero)	현재 행의 가장 맨 앞으로 이동합니다.
\$	현재 행의 가장 맨 끝으로 이동합니다.
gg	문서의 가장 처음으로 이동합니다.
G	문서의 가장 끝으로 이동합니다.

명령 모드

복사 및 삭제와 관련된 명령어 목록입니다.

명령어		설명
삭제	x	현재 커서에 해당하는 글자를 삭제합니다.(delete 개념)
삭제	X	현재 커서의 앞 1글자를 삭제합니다.(backspace 개념)
삭제	dd	1줄을 삭제합니다.
삭제	숫자dd	현재 커서로부터 숫자 만큼의 행을 삭제합니다.
삭제	u	이전 상태로 되돌리기(ctrl + z 개념) 단, 작업한 양이 많다면 초기 상태로 되돌리지 못할 수 있습니다.
삭제	U	undo와 redo가 토글이 됩니다.
복사	yy	현재 커서가 있는 라인을 복사(ctrl + c 개념)
복사	숫자yy	입력한 숫자 만큼 행을 복사합니다.
복사	p	현재 행의 아래에 붙여 넣기를 수행합니다.
복사	P	현재 행의 위에 붙여 넣기를 수행합니다.

≡ 명령 모드

검색하기와 관련된 명령어 모음입니다.

명령어	설명
/ugcadman 엔터	ugcadman이라는 단어를 검색합니다.
n	다음(next) 단어 검색을 위하여 누르면 됩니다.
N	n의 반대입니다.(previous)
비고	?ugcadman : ?는 /와 달리 문서의 가장 끝에서부터 검색합니다.

≡ 명령 모드

종료하기와 관련된 명령어 모음입니다.

명령어	설명
:q	현재 작업 중인 문서를 저장하지 않은 상태에서 빨간 메시지 경고 박스가 뜬다. 저장 또는 강제 종료 중 선택하라는 의미이다. No write since last change(add ! to override)
:q!	현재 작업 중인 문서를 강제로 종료합니다.
:wq	현재 작업 중인 문서를 저장한 다음 종료합니다.
:w	현재 작업 중인 문서를 저장합니다.

≡ 입력 모드

입력 모드는 버퍼에 내용을 입력할 수 있는 모드입니다.

입력 모드

명령 상태에서 a, i, o 등의 키보드를 누르면 진입할 수 있습니다.

하단에 -INSERT- 라고 표시가 됩니다.

R을 누른 경우에는 -REPLACE- 라고 표시됩니다.

≡ 입력 모드

입력 모드로 진입하기 위해서 다음과 같은 키보드들을 사용할 수 있습니다.

명령어	설명
키보드 i 입력	현재 위치에서 insert 모드로 바뀝니다. 입력하면 현재 커서의 왼쪽에 글자가 입력됩니다.
키보드 I 입력	현재 행의 제일 처음 위치에서 입력 모드로 변경됩니다.
키보드 a 입력	현재 커서의 오른쪽 1칸으로 이동후 입력 모드가 됩니다. i, o 중에서 편한 것을 사용하면 됩니다.
키보드 A 입력	현재 행의 제일 마지막 위치에서 입력 모드로 변경됩니다.
키보드 o 입력	현재 위치의 아래에 빈 줄 1개를 생성후 입력 모드로 전환합니다.
키보드 s 입력	현재 문자를 지우고 입력 모드로 전환합니다.
키보드 S 입력	현재 행의 모든 문자를 지우고 입력 모드로 전환합니다.

≡ 실행 모드

`검색 기능`과 관련된 실행 모드 명령어는 다음과 같습니다.

명령어	설명
/Pattern 엔터	Pattern 을 검색
?Pattern 엔터	Pattern 을 검색

검색 기호

항목	설명
기호 /	끝까지 검색이 되고 나면 다시 순환하여 검색합니다. n(아래 방향으로 다음 검색), N(위 방향으로 다음 검색)
기호 ?	N(아래 방향으로 다음 검색), n(위 방향으로 다음 검색)

실행 모드

‘치환 기능’과 관련된 실행 모드 명령어는 다음과 같습니다.

명령어	설명
<code>:[범위]s/[Old]/[New]/[옵션]</code>	Old를 New로 치환합니다.

부가 설명

n은 라인 번호를 의미하는 정수형 숫자입니다.

범위는 n 혹은 n,n 혹은 기호 %의 형식으로 사용할 수 있습니다.

g 옵션을 주면 적용되는 라인의 모든 부분을 변경합니다.

g 옵션을 주지 않으면 처음 찾은 부분만 변경합니다.

치환 작업이 의도와 맞는지 확인하려면 `c` 옵션을 추가해 확인 메시지를 띄울 수 있습니다.

사용 예시

항목	설명
<code>:10s/Yesterday/morning/</code>	10 번째 줄에 있는 ‘Yesterday’를 ‘morning’으로 치환합니다.
<code>:%s/Yesterday/morning/g</code>	모든 ‘Yesterday’를 ‘morning’으로 치환합니다. %s는 전체 파일의 매칭 항목에 적용이 됩니다.
<code>:2,5s/Yesterday/morning/gc</code>	2번 줄부터 5번 줄까지 모든 ‘Yesterday’를 ‘morning’으로 치환하되, 사용자에게 y/n 형식으로 물어 봅니다.

≡ 명령 모드

치환하기와 관련된 명령어 모음입니다.

명령어	설명
shift + r	특정 글자를 치환하고자 하는 경우 단어에 커서를 두고 키보드 shift + r을 입력합니다. 바꾸고자 하는 단어를 입력하게 되면 치환이 됩니다.
:[n] /old/new	old라는 단어를 new라는 단어로 치환합니다. n은 줄번호를 의미합니다.
:3s /ugcadman/linux	3행의 첫번째 ugcadman이라는 단어를 linux으로 변경하라.
:3s /ugcadman/linux/g	특정 단어 전부를 변경하는 경우 /g를 사용하라.
:3,4s /ugcadman/linux/g	3행과 4행을 모두 치환하라.
:%s /ugcadman/linux/g	문서 전체에서 치환하라.

≡ 실행 모드

`Shell 명령어`와 관련된 명령어는 다음과 같습니다.

느낌표 기호는 vi 모드에서 잠시 외부 linux 명령어를 실행하기 위한 명령어입니다.

명령어	설명
:! [command]	vi를 잠시 중단하고 명령어를 수행합니다. 엔터 키를 눌러서 vi 편집기로 복귀합니다.
..! [command]	수행한 명령의 결과를 vi 편집기로 출력합니다. 출력된 내용을 저장하려면 `:wq sample.txt` 형식으로 저장이 됩니다.

≡ 실행 모드

`창 분할 하기` 기능과 관련된 명령어는 다음과 같습니다.

명령어	설명
:split 또는 :sp	수평으로 창을 분할합니다.
:vsplit 또는 :vs	수직으로 창을 분할합니다.

관련 명령어

항목	설명
:quit	활성화된 현재 창을 닫습니다.
ctrl + 또는 ctrl -	글자를 크게 또는 작게 보기 위하여 창의 크기를 조정합니다.
ctrl w	다른 창으로 위치 이동을 합니다.

≡ 실행 모드

‘파일’ 및 ‘실행’과 관련된 실행 모드 명령어는 다음과 같습니다.

명령어	설명
:e some.txt	some.txt 이라는 파일을 엽니다.(파일 불러 오기)
:enew	현재 작업 중인 파일을 닫고 빈 문서를 엽니다.(새 문서 작성하기)
:q	변경된 내용이 없는 경우 종료합니다.
:q!	변경된 내용이 있어도 무시하고 강제 종료 합니다.
:w	파일 저장
:wq	파일 저장 후 종료합니다.
:w >> filename	현재 작업 중인 내용을 filename 이라는 파일에 저장하겠습니다.
:f	현재 작업 중인 문서의 정보(파일의 이름, 라인 수)를 확인합니다.
:[n]r[filename]	filename 파일의 내용을 현재 편집 중인 파일의 n 라인부터 삽입합니다.
:[n]r![command]	Command 실행 결과를 파일의 n 라인부터 삽입
:777	해당 줄번호로 이동합니다.

≡ 실행 모드

기타 실행 모드 명령어는 다음과 같습니다.

명령어	설명
:set ts=4	Tab 사이즈 조절
:set nu	현재 문서에 대하여 Line number를 보여 줍니다.
:set nonu	현재 문서에 대하여 Line number를 보여 주지 않습니다.
:nohl	High light 비활성화

리눅스(Linux)

Ownership&Permissions



Ownership & Permissions

`사용자`(`user`)와 `그룹`(`group`)

리눅스는 1대의 컴퓨터에 여러 명이 동시 접속 가능한 **다중 사용자 시스템**입니다.

VideoTeam



김지훈



윤하늘

Anchor



박서준



조기현

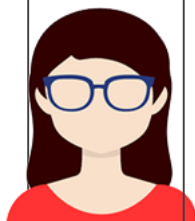
Editorial



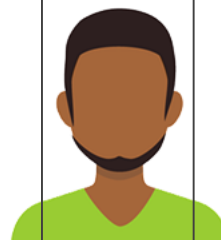
강민수



최유진



이지은



장민석

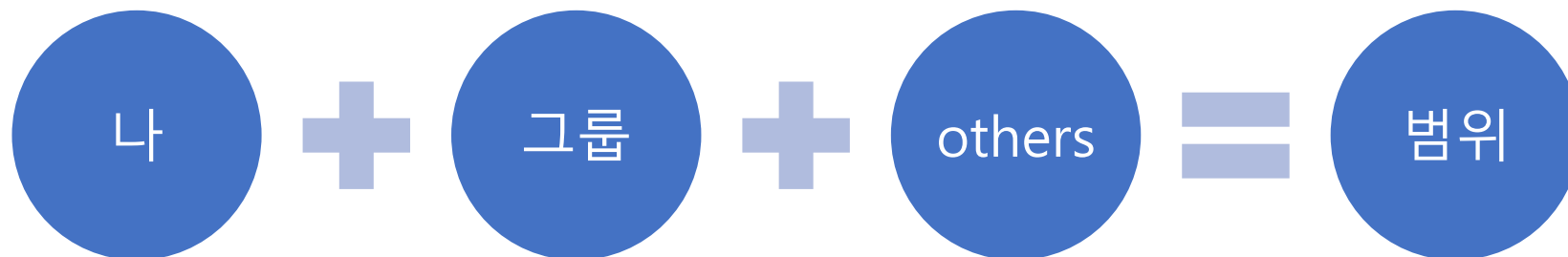
Ownership&Permissions

소유권과 퍼미션

소유권과 퍼미션(permission)은 시스템에서 **파일과 디렉토리에 대한 접근 권한을 관리**하는 중요한 개념입니다

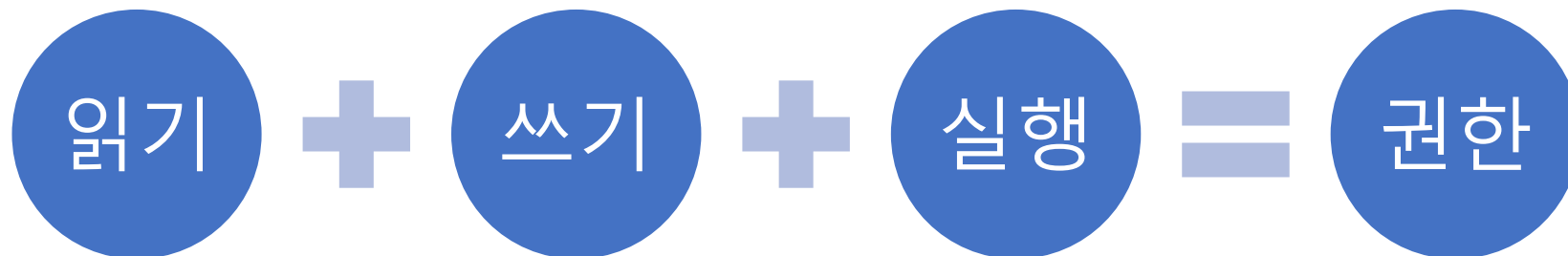
소유권

특정 파일이나 디렉토리의 소유자(owner)가 누구인지를 의미합니다.



퍼미션

파일에 대해 읽기, 쓰기, 실행을 할 수 있는 권한을 의미합니다.



Ownership&Permissions

Ownership&Permissions 개요

파일과 디렉토리는 소유권(Ownership)과 권한(Permissions)을 가지고 있습니다.

Ownership&Permissions에는 파일 유형, 파일 허가권, 링크 수, 파일 소유자 이름, 파일 소유 그룹 이름 등의 구성 요소가 있습니다.

소유권(Ownership)

리눅스 파일 시스템에서는 각 파일과 디렉토리에 대해 다음과 같은 소유권이 지정됩니다.

항목	설명
소유자(Owner)	파일을 생성한 사용자로, 파일에 대한 기본적인 접근 권한을 가집니다.
그룹(Group)	파일이 속한 그룹으로, 그룹에 속한 사용자들이 파일에 접근할 수 있도록 합니다.
기타 사용자(Others)	소유자와 그룹에 속하지 않는 모든 사용자입니다.

소유권 확인

‘소유권’을 확인하려면 `ls -l` 명령어를 사용합니다.

예시에서 ‘kbs7’의 소유자는 ‘ugcadman’입니다.

```
total 16
drwxr-xr-x 1 ugcadman 197121  0 May 31 10:25 ./
drwxr-xr-x 1 ugcadman 197121  0 May 30 17:40 ../
drwxr-xr-x 1 ugcadman 197121  0 May 30 15:07 kbs7/
```

Ownership&Permissions

Ownership&Permissions 개요

파일과 디렉토리는 소유권(Ownership)과 권한(Permissions)을 가지고 있습니다.

권한(Permissions)

리눅스에서 파일과 디렉토리의 권한은 소유자, 그룹, 기타 사용자 등에 대하여 세 가지로 구성됩니다.

항목	권한	설명
파일	읽기(Read)	파일 내용을 읽을 수 있는 권한.(cat , more , vim , cp 등과 연관)
	쓰기(Write)	파일 내용을 수정할 수 있는 권한.(cat , echo , vim 등과 연관) vim 같은 경우는 :wq!로 저장이 가능합니다.(소유주만 적용)
	실행(eXecute)	파일의 실행 가능 여부와 관련이 있는 권한
디렉토리	읽기(Read)	ls , dir등과 같은 디렉토리의 내부 내용을 읽어 들이는 것과 관련이 있다.
	쓰기(Write)	mkdir , touch , rm , mv등과 같은 디렉토리 내부에 생성,삭제와 관련이 있다
	실행(eXecute)	디렉토리의 접근과 연관이 있습니다. 디렉토리에 x가 없으면 대부분의 명령어를 실행할 수 없습니다.

권한 확인

‘권한’을 확인하려면 ls -l 명령어를 사용합니다.

권한은 10자리 문자열로 표시되는 데, 예시에서 kbs7 디렉토리의 권한은 ‘drwxr-xr-x’ 입니다.

```
total 16
drwxr-xr-x 1 ugcadman 197121    0 May 31 10:25 ./
drwxr-xr-x 1 ugcadman 197121    0 May 30 15:07 kbs7/
```

다음 페이지에 계속...

Ownership&Permissions

Ownership&Permissions 개요

권한 정보를 확인하는 방법을 살펴 보겠습니다.

권한 문자열

‘권한’과 관련된 문자열은 10자리 형식의 문자열로 표시되어 있습니다.

첫 번째 문자는 파일 유형을 나타내며 ‘-’ 기호는 일반 파일, ‘d’ 기호는 디렉토리를 의미합니다.

나머지 9자리는 소유자, 그룹, 기타 사용자에게 대한 권한을 나타냅니다.

각 그룹에 대해 r, w, x 순서로 나옵니다.

```
total 16
drwxr-xr-x 1 ugcadman 197121  0 May 31 10:25 ./
drwxr-xr-x 1 ugcadman 197121  0 May 30 17:40 ../
drwxr-xr-x 1 ugcadman 197121  0 May 30 15:07 kbs7/
-rw-r--r-- 1 ugcadman 197121 1298 May 31 10:25 vitest.txt
-rw-r--r-- 1 ugcadman 197121 7208 May 30 15:42 java.txt
```

파일/디렉토리 구분자	Owner			Group			Other		
d	r	w	x	r	w	x	r	w	x
	4	2	1	4	2	1	4	2	1
	7			7			7		

퍼미션

항목	심벌	숫자
읽기	r	$2^2 = 4$
쓰기	w	$2^1 = 2$
실행	x	$2^0 = 1$

권한 종류	설명
없음	0
실행(x)	1
쓰기(w)	2
쓰기(w) + 실행(x)	3
읽기(r)	4
읽기 + 실행	5
읽기 + 쓰기	6
읽기 + 쓰기 + 실행	7

다음 페이지에 계속...

Ownership&Permissions

Ownership&Permissions 개요

관련된 구성 요소는 다음과 같은 항목들 있습니다.

파일/디렉토리 권한

항목	권한	설명
파일	r	cat, more, vim, vi, cp 등과 같은 파일의 내용을 읽어 들이는 것과 관련이 있습니다.
파일	w	cat, echo, vi, vim 등과 같은 파일의 내용을 수정,변경 하는 것과 관련이 있습니다. vim 같은 경우는 :wq!로 저장이 가능하긴 합니다.(소유주만 적용)
파일	x	실행 파일 같은 경우 실행 여부와 관련이 있습니다.(shell 파일)
디렉토리	r	ls, dir등과 같은 디렉토리의 내부 내용을 읽어 들이는 것과 관련이 있습니다.
디렉토리	w	mkdir, touch, rm, mv등과 같은 디렉토리 내부에 생성,삭제와 관련이 있습니다.
디렉토리	x	디렉토리의 접근과 관련이 있습니다. 디렉토리에 x가 없다는것은 대부분의 명령어를 이용 할수가 없게 됩니다.

Ownership&Permissions

`chmod 명령어

chmod 명령어는 파일이나 디렉토리의 권한(permission)을 변경하기 위한 명령어입니다.

chmod 명령어는 기호 모드(Symbolic Mode)와 숫자 모드(Numeric Mode)로 사용할 수 있습니다.

기호 모드 사용 형식

```
chmod [누구][연산자][권한] 파일/디렉토리
```

사용 옵션

항목	설명
누구	u(소유자), g(그룹), o(기타 사용자), a(모든 사용자).
연산자:	+(추가), -(제거), =(설정).
권한	r(읽기), w(쓰기), x(실행).

사용 예시

예시	설명
chmod u+x script.sh	소유자에게 실행 권한 추가
chmod g-w report.txt	그룹에서 쓰기 권한 제거
chmod o=r file.txt	기타 사용자에게 읽기 권한만 설정
chmod g+rx test	test 파일의 그룹에 읽기와 실행 권한을 허가(symbolic method)

Ownership&Permissions

chmod 명령어

chmod 명령어는 파일이나 디렉토리의 권한(permission)을 변경하기 위한 명령어입니다.

숫자 모드 사용 형식

여기서 XXX는 3자리 숫자 형식으로 소유자, 그룹, 기타 사용자에게 대한 권한입니다.

```
chmod [옵션] XXX 파일/디렉토리
```

사용 예시

예시	설명
chmod 755 myfile	소유자는 rwx(7), 그룹은 r-x(5), 기타 사용자는 r-x(5)
chmod 644 myfile	소유자는 rw-(6), 그룹은 r--(4), 기타 사용자는 r--(4)
chmod 777 myfile	모든 사용자에게 rwx 권한 부여

Ownership&Permissions

`chown` 명령어

소유권을 변경하고자 하는 경우 사용하는 명령어입니다.

"change owner"의 줄인 말로써, 일반적으로 시스템 관리자나 파일 소유자가 파일의 소유자를 변경할 때 사용됩니다.

사용 구문

```
chown [옵션] 소유자:그룹 파일명
```

사용 옵션

항목	설명
-R	지정된 파일 및 디렉토리를 재귀적으로 변경합니다.
-v	변경 내용을 자세히 출력합니다.
-c	변경 내용을 출력합니다.

사용 예시

예시	설명
chown user:group example.txt	파일 'example.txt'의 소유자를 'user'로, 그룹을 'group'으로 변경합니다.
chown -R user:group directory_name	디렉토리 및 그 하위 항목을 재귀적으로 변경하려면 -R 옵션을 사용할 수 있습니다.
chown testuser1 test	test 파일의 소유자를 testuser1으로 변경합니다.
chown testuser1:testuser1 test	test 파일의 소유자와 그룹을 testuser1으로 변경합니다.
chown testuser1:root test	test 파일의 소유자는 testuser1 그룹은 root로 변경합니다

Ownership&Permissions

`chgrp` 명령어

파일이나 디렉토리의 소유자(owner)와 그룹(group) 소유권을 변경하는 데 사용되는 명령어입니다.

"chgrp"는 "change group"의 줄임말입니다.

사용 구문

chgrp [옵션] 그룹명 파일 또는 디렉토리

사용 옵션

항목	설명
-R	지정된 디렉토리 및 해당 하위 디렉토리 및 파일에 대한 작업을 재귀적으로 수행합니다.
-v	변경 내용을 자세히 출력합니다.

사용 예시

예시	설명
chgrp developers example.txt	파일 "example.txt"의 그룹 소유권을 "developers" 그룹으로 변경합니다.
chgrp -R admins project	재귀적으로 디렉토리 "project"의 모든 파일과 하위 디렉토리의 그룹 소유권을 "admins" 그룹으로 변경합니다.
chgrp testgroup1 test	test 파일의 소유그룹 을 testgroup1으로 변경합니다.

리눅스(Linux)

Shell

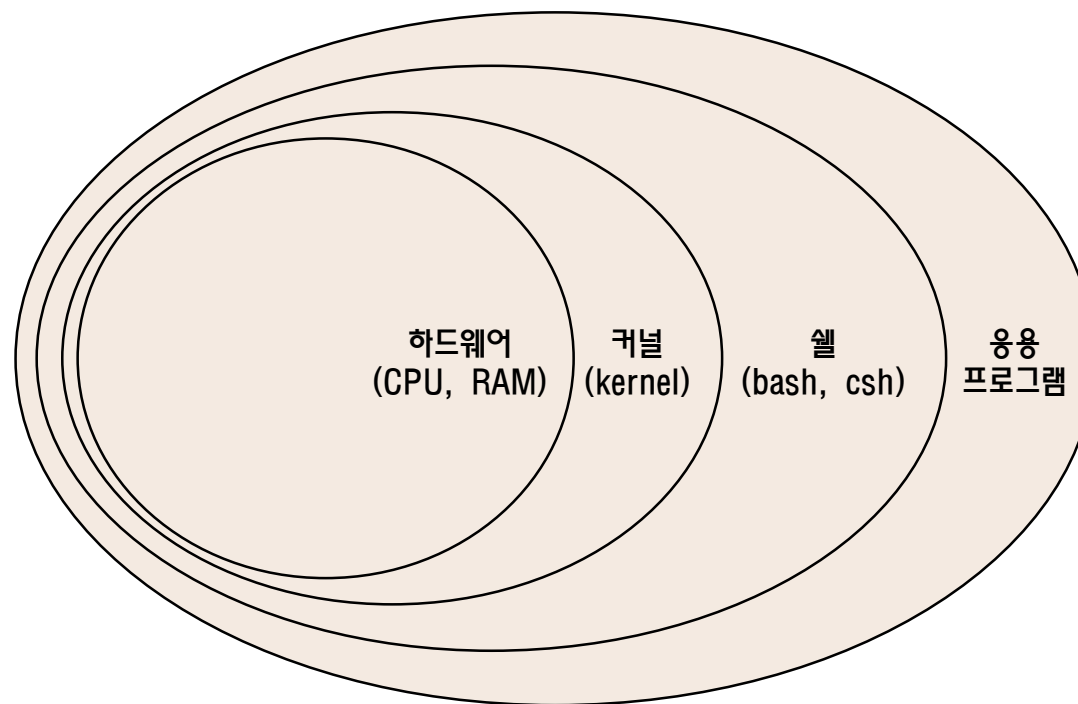


≡ Where is Shell?

Shell은 사용자와 운영 체제 사이의 인터페이스 역할을 합니다.

Shell의 주요 역할

- 명령어 해석 및 실행
- 프로그램 실행
- 스크립트 실행
- 환경 변수 관리
- 파일 및 디렉토리 관리
- 프로세스 관리
- 입출력 리다이렉션 및 파이프 처리



≡ What is Shell?

Shell은 사용자와 운영 체제 사이의 인터페이스 역할을 합니다.

리눅스에서 사용자가 입력하는 명령어를 해석하고, 운영 체제가 이를 실행하도록 전달합니다.

Shell 명령어는 리눅스에서 파일 관리, 프로세스 제어, 네트워크 설정 등 시스템 작업을 수행하는 데 사용됩니다.

주요 특징

항목	설명
명령어 기반	키보드로 명령을 입력하여 작업을 수행.
스크립트 가능	Shell 스크립트를 작성하여 반복 작업을 자동화.

Shell의 종류

cshell, bourneshell, bashshell, tcshell, kornshell, 등 다양한 종류가 있습니다.

항목	설명
bash(Bourne Again Shell)	대부분의 Linux 배포판에서 기본적으로 사용.
zsh	사용자 친화적이고 강력한 기능 제공.
sh(Bourne Shell)	기본적인 명령어 처리.
ksh(Korn Shell)	고성능과 강력한 스크립트 기능 제공.
기타 셸	csh(C Shell), 등.

≡ Shell의 역할

운영 체제의 핵심인 커널과 사용자를 연결하는 중간 계층으로, 명령어를 해석하고 실행하는 데 중요한 역할을 합니다.

Shell의 주요 역할

항목	설명	사용 예시
명령어 해석 및 실행	명령어를 해석하여 운영 체제가 이해할 수 있는 형태로 변환하고 실행합니다.	<code>ls -l</code>
프로그램 실행	파일 탐색, 프로그램 실행, 프로세스 관리 등 다양한 작업을 수행합니다.	<code>python3 script.py</code>
스크립트 실행	Shell 스크립트를 통해 반복 작업을 자동화하고, 여러 명령어를 순서대로 실행할 수 있습니다.	<pre># sample.sh echo "Hello, World!" mkdir test_directory</pre> 예시 <code>bash sample.sh</code> 명령어를 통해 스크립트 파일이 실행됩니다.
환경 변수 관리	시스템 환경 변수(예: PATH, HOME)를 설정 및 관리합니다. 이를 통해 프로그램과 사용자 설정을 유지합니다.	<code>echo \$HOME</code>
파일 및 디렉토리 관리	파일 생성, 삭제, 이동, 복사 등의 작업을 수행할 수 있습니다.	<pre>touch myfile.txt # 파일 생성 mv myfile.txt /tmp # 파일 이동</pre>
프로세스 관리	실행 중인 프로세스를 모니터링하고 제어할 수 있습니다.	<pre>ps aux # 현재 실행 중인 프로세스 확인 kill -9 1234 # 특정 프로세스 종료</pre>
입출력 리다이렉션 및 파이프 처리	Shell은 명령어의 입력과 출력을 제어할 수 있습니다.	<pre>ls > files.txt # 출력 리다이렉션 cat file1 grep "text" # 파이프 처리</pre>

≡ Bash Shell

bash shell은 bourne shell을 기반으로 c shell과 korn shell의 장점들을 합친 shell입니다.

bash의 주요 특징

- 다양한 내장 명령어
- Alias 기능(명령어 별칭 기능)
- 히스토리 기능(방향 키 ↑ 과 ↓)
- 연산 기능
- Job Control 기능
- 자동 이름 완성 기능(Tab 키)
- Prompt 제어 기능
- 명령 편집 기능

≡ 환경 변수(Environment Variable)

운영 체제와 실행 중인 프로그램이 정보를 교환하거나 동작을 제어하는 데 사용하는 키-값 쌍입니다.

환경 변수는 시스템이나 사용자 환경에 영향을 주는 중요한 설정 값을 담고 있습니다.

bash 주요 환경 변수

항목	설명
ENV	.bashrc 파일에서 설정하며 함수와 별명 등을 설정하는 환경 파일의 이름을 설정
EUID	Shell이 시작할 현재 사용자의 유효 ID를 확장합니다.
HISTFILE	명령어 히스토리를 저장할 파일이름. 기본값은 ~/.bash_history이다.
HOME	사용자의 홈 디렉토리의 값을 가지고 있으며 특정 디렉토리를 지정하지 않고 cd 명령어를 사용할 때 이용합니다.
LANG	LC_로 시작하는 변수들에 포함되지 않는 로케일을 설정
PATH	명령어의 검색 경로, 콜론으로 구분한 디렉토리 목록
PPID	부모프로세스의 PID
PWD	현재 작업 디렉토리
SHELL	현재 사용되어지고 있는 SHELL
UID	현재 사용자의 UID

≡ Shell Script 언어

≡ for 구문

특정 내용을 반복하기 위하여 for 구문을 사용합니다.

사용 형식

```
for 변수 in 복수; do
    반복할 내용
done
```

```
for user in $(awk -F: '($3 >= 1000 {print $1})' /etc/passwd)
do
    echo "$user: $(groups $user)"
    echo "-----"
done
```

실행 결과

```
nobody : nobody : nogroup
-----
broadcast : broadcast : broadcast adm cdrom sudo dip plugdev lxd
-----
skywalker : skywalker : skywalker
-----
sunnyday : sunnyday : sunnyday journalist
-----
rainbow7 : rainbow7 : rainbow7
-----
pikachu : pikachu : pikachu
-----
```

리눅스(Linux)

File System



File System 개요

파일 시스템은 디스크 위의 데이터를 파일과 디렉토리 형태로 관리하는 규칙입니다.

파일 시스템(File System)

파일 시스템은 파일, 디렉토리, 그리고 그와 관련된 메타 데이터를 구성하고 관리하는 데 사용이 됩니다.
운영 체제가 저장 장치(예: 하드 디스크, SSD, USB 드라이브 등)에 접근할 수 있도록 도와 줍니다.

파일 시스템의 필요성

디스크는 단순히 데이터를 저장하는 공간일 뿐입니다.
파일 시스템이 없으면 파일의 위치를 알 수 없습니다.
파일 이름, 크기, 생성 시간 관리가 불가능합니다.
사람이 데이터를 쉽게 사용하도록 도와주는 체계입니다.

무엇을 관리해주나

파일 이름
파일 크기
저장 위치
생성 / 수정 / 접근 시간
권한 정보
소유자 정보

File System 개요

파일 시스템은 디스크 위의 데이터를 파일과 디렉토리 형태로 관리하는 규칙입니다.

주요 특징

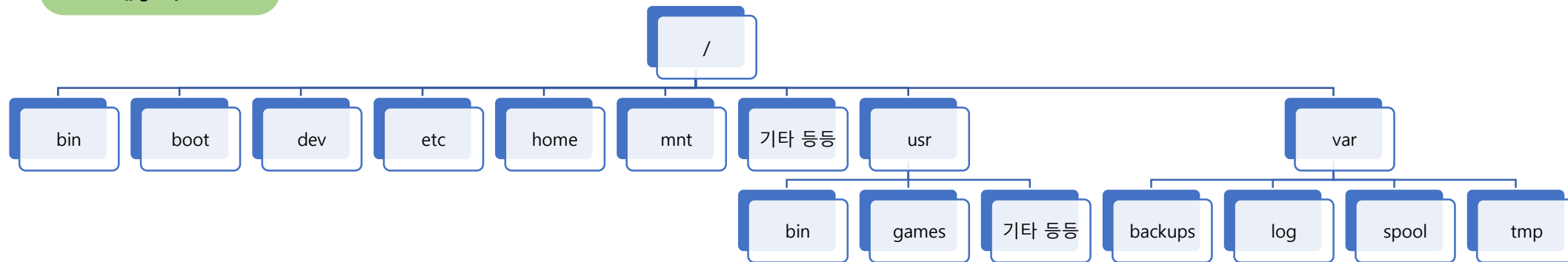
항목	설명
트리 구조	리눅스 파일 시스템은 루트 디렉토리(`/`)를 최상위로 하는 트리 구조를 가집니다. 모든 파일과 디렉토리는 `/`에서 시작됩니다. 예시) `/home/user/documents`
파일과 디렉토리	리눅스에서 파일은 일반 파일, 디렉토리, 심볼릭 링크 등 다양한 종류로 구분됩니다. 일반 파일: 텍스트 파일, 바이너리 파일 등. 디렉토리: 파일과 디렉토리를 포함하는 컨테이너. 심볼릭 링크: 다른 파일의 경로를 참조하는 파일.
마운트(Mount)	리눅스에서는 파일 시스템을 특정 디렉토리(마운트 포인트)에 연결해야 접근할 수 있습니다. 예시) `/dev/sda1`를 `/mnt`에 마운트하면 `/mnt`를 통해 해당 저장 장치의 파일에 접근 가능.
유니파이드 파일 시스템	리눅스에서는 모든 장치(하드 디스크, 네트워크 드라이브 등)가 단일 파일 트리에 통합되어 관리됩니다.

File System

File System 개요

주요 디렉토리의 계층 구조는 다음과 같습니다.

계층 구조



File System 개요

주요 디렉토리의 계층 구조는 다음과 같습니다.

주요 디렉토리

디렉토리	설명
/	리눅스 상에 존재하는 모든 파일과 디렉토리의 최상위에 위치하는 디렉토리입니다.
/boot	리눅스 커널의 메모리 이미지와 부팅 과정에서 필요한 정보 파일들이 포함되어 있습니다.
/bin	기본적인 명령어들이 포함되어 있습니다.
/root	슈퍼 유저와 root의 홈 디렉토리입니다.
/home	관리자 이외의 일반 사용자들의 홈 디렉토리가 생성되는 위치입니다.
/etc	설정 파일을 저장하는 디렉토리입니다. 시스템이나 사용자 관리를 위한 자료 파일이나 관리자 용 명령어들을 저장하고 있습니다.
/dev	장치(device) 파일이라고 하는 파일 시스템과 하드웨어 간의 인터페이스를 담당하는 파일이 저장되어 있습니다. 예시) /dev/modem, /dev/console 등등
/lib	각종 프로그래밍 언어들을 위한 라이브러리 저장소입니다.

File System 개요

주요 디렉토리의 계층 구조는 다음과 같습니다.

주요 디렉토리

디렉토리	설명
/mnt	원격 장치(remote device)를 이용하기 위한 디렉토리입니다.
/media	로컬 장치(local device)를 이용하기 위한 디렉토리입니다.(CDRom)
/sbin	시스템 운영 및 관리자를 위한 명령어들이 포함되어 있습니다. 부팅 과정에서 필요한 명령어들은 여기에 포함되어 있고, 정상 동작 상태에서 필요한 크기가 큰 파일들은 /usr/sbin 디렉토리에 포함되어 있습니다.
/usr	루트 파일 시스템과 구조는 유사하고 용량이 크고 자주 사용되지 않는 파일들이 포함되어 있습니다. 윈도우의 Program File 폴더와 유사한 개념입니다.
/proc	커널과 프로세스 정보를 얻을 수 있는 가상 파일 시스템입니다. 관리자는 이 디렉토리에 대하여 잘 알고 있어야 합니다.
/tmp	잠시 사용되는 임시 파일들을 위한 디렉토리입니다. 시스템 재시작시 모든 내용이 사라집니다.
/var	/var/log 또는 /var/adm과 같이 자주 변경이 되는 시스템 파일들이 포함되어 있습니다. log는 외부에서 접근하는 기록 정보인데, 로그 서버를 따로 만들어서 관리하기도 합니다.

파일 시스템의 역할

리눅스 파일 시스템은 데이터를 저장하고, 시스템을 운영하며, 보안을 담당하는 핵심 요소입니다.

데이터 저장

사용자 파일(문서, 이미지, 코드)
시스템 파일(설정 파일, 실행 파일)
로그 파일, 임시 파일 등

중개자 역할

하드디스크 / SSD와 사용자 명령어 사이를 연결
사용자는 복잡한 디스크 구조를 몰라도 파일 이름만으로 접근 가능

시스템 운영의 기반

리눅스는 파일 기반 운영 체제
설정, 장치, 프로세스 정보까지 파일로 관리
"모든 것은 파일이다"라는 철학

보안과 접근 제어

사용자별 접근 권한 관리
읽기 / 쓰기 / 실행 권한 제공
시스템 보호 역할 수행

파일 시스템과 운영 체제의 관계

파일 시스템은 운영 체제가 디스크를 제어하고 사용자에게 파일 서비스를 제공하는 핵심 연결 고리입니다.

운영 체제의 역할

하드웨어 자원 관리
프로세스 관리
메모리 관리
파일 시스템 관리

운영 체제와 파일 시스템의 연결

운영 체제는 파일 시스템을 통해 디스크에 접근
사용자가 ls, cd, cat 같은 명령어를 입력하면 운영 체제가 파일 시스템을 통해 처리

사용자 관점에서의 파일 시스템

사용자는 디스크 구조를 직접 다루지 않음
파일 시스템이 제공하는 추상화된 구조만 사용

시스템 안정성과 파일 시스템

파일 시스템 오류 → 시스템 부팅 실패 가능
안정적인 파일 시스템은 운영 체제의 안정성과 직결

File System

File System의 종류

리눅스에서 사용하는 대표적인 파일 시스템의 종류에는 다음과 같은 항목들이 있습니다.

File System의 종류

항목	설명
EXT 계열	EXT2 초창기 리눅스 기본 파일 시스템, 저널링 없음. EXT3 EXT2에 저널링 추가, 안정성과 데이터 복구 성능 향상. EXT4 대용량 저장소 지원, 성능과 안정성 향상.
XFS	고성능과 대용량 파일 처리에 강점. 데이터 손실을 줄이는 저널링 지원.
Btrfs	최신 파일 시스템, 스냅샷, 압축, RAID 등 고급 기능 제공. 안정화 단계.
FAT 및 NTFS	Windows와의 호환성을 위해 사용. USB 드라이브 등에서 일반적으로 FAT32나 exFAT 사용.
ZFS	데이터 무결성 보장, 스냅샷, 압축 등 제공. 고성능과 안정성을 목표로 설계.

≡ File System 관리 명령어

File System 관리를 위한 명령어는 다음과 같은 항목들이 있습니다.

File System 명령어

항목	설명	예시
파일 시스템 확인 및 생성	mkfs 명령어는 새로운 파일 시스템을 생성합니다.	mkfs.ext4 /dev/sda1
	fsck 명령어는 파일 시스템의 무결성 확인 및 복구를 위하여 사용합니다.	
마운트 및 언마운트	mount 명령어는 파일 시스템을 마운트하기 위하여 사용합니다.	mount /dev/sda1 /mnt
	umount 명령어는 마운트 해제하고자 할 때 사용합니다.	
현재 마운트 상태 확인	df 명령어는 파일 시스템의 디스크 사용량을 확인합니다.	
	mount 명령어는 또는 `/etc/fstab` 파일을 이용하여 마운트된 파일 시스템을 확인할 수 있습니다.	

File System 생성하기

`mkfs나 mke2fs 명령어를 사용하면 다양한 파일 시스템 유형에 대해 파일 시스템을 생성할 수 있습니다.

디스크 파티션이나 디스크 이미지에 새로운 파일 시스템을 만듭니다.

다양한 파일 시스템 유형을 지원하며, 특정 파일 시스템에 대한 하위 명령을 사용하여 작업을 수행합니다.

각 파일 시스템 유형마다 특정 명령과 옵션들이 있습니다.

사용 구문

```
mkfs [ option ] [ Device Name ]
```

사용 옵션

옵션	설명
-c	파일 시스템 생성시 배드 블록을 체크합니다.
-t [Filesystem]	만들어질 파일 시스템의 종류를 지정합니다.
-v	파일 시스템을 생성하는 과정을 자세히 보여 줍니다.

File System 생성하기

`mkfs나 mke2fs 명령어를 사용하면 다양한 파일 시스템 유형에 대해 파일 시스템을 생성할 수 있습니다.

사용 예시

```
mkfs -t ext2 /dev/sdb1  
mkfs -t ext3 /dev/sdb2  
mke2fs /dev/sdb1  
mke2fs -j /dev/sdb2
```

축약된 형식

```
sudo mkfs.ext4 /dev/sdb  
sudo mkfs -t ext4 /dev/sdb
```

mkfs 명령어를 이용하여 ext2, ext3 파일 시스템을 생성합니다.

mke2fs 명령어를 이용하여 ext2, ext3 파일 시스템을 생성합니다.

What is `Mount`?

파일 시스템 구조 내에 있는 일련의 파일들을 사용자나 사용자 그룹들이 이용할 수 있도록 만드는 동작을 의미합니다.

다양한 옵션과 함께 사용되어 특정 디바이스나 네트워크 리소스를 특정 디렉토리에 연결할 수 있습니다.

또한, mount 명령은 현재 마운트된 파일 시스템의 목록을 확인하는 데도 사용됩니다.



로컬 디스크 (C:)



새 볼륨 (D:)



ugcadman70 (E:)

파티션을 생성하면 자동으로 "드라이브 이름"이 할당됩니다.
C 드라이브, D 드라이브 등등



mount

Unix나 Linux 운영 체제는 관리자가 직접 파티션을 특정 문자(디렉토리)에 연결시켜 주어야 합니다.

항목	설명
파티션 연결	mount 명령어 /etc/fstab 파일 사용하기
파티션 연결 해지	umount 명령어

What is `Mount`?

파일 시스템 구조 내에 있는 일련의 파일들을 사용자나 사용자 그룹들이 이용할 수 있도록 만드는 동작을 의미합니다.

마운트(Mount)

Windows 시스템에서는 파티션을 생성하면 자동으로 "드라이브명" 이라고 하는 특정 문자에 자동으로 연결되어 있지만 Unix/Linux는 관리자가 직접 파티션을 특정 문자(디렉토리)에 연결하여야 하는데 그것을 "마운트"라고 합니다.

Mount의 특징

리눅스 환경에서 "mount"라는 명령은 논리적으로 디스크와 디렉토리를 연결하는 것을 의미합니다.
지역이나 원격으로 행해질 수 있으며, 지역 마운트는 하나의 논리적 시스템으로서 행동하도록 여러대의 디스크 드라이브들을 하나의 컴퓨터 상에 접속합니다.
원격 마운트는 다른 컴퓨터 상의 디렉토리들에 접속하기 위해 NFS(Network File System)을 사용합니다.

Mount와 UnMount

리눅스에서는 파티션을 연결하는 두 가지 방법은 1번째 mount 명령어를 통해 수동으로 연결하는 방법이 있습니다.
2번째 방법은 /etc/fstab 파일 내에 마운트할 항목을 추가하여 부트시에 자동으로 마운트 하는 방법이 있습니다.
파일 시스템의 사용을 중지할 때는 umount 명령어를 사용합니다.

Mount 명령어 사용 형식

Mount 명령어의 사용 방법은 다음과 같습니다.

사용 구문

```
mount [ Option ] [ Device ] [ Directory ]
```

사용 옵션

옵션	설명
-a	/etc/fstab에 기술되어있는 모든 파일 시스템을 마운트 합니다.
-t <type>	파일 시스템 유형을 지정합니다(예: ext4, xfs, vfat).
-n	/etc/mtab 파일에 정보를 남기지 않고 마운트합니다.
-l	현재 마운트된 파일 시스템의 UUID를 표시합니다.
-o	-o [Option] 뒤에 콜론(":") 으로 다음 옵션들을 사용할 수 있습니다. async : 파일 시스템에 대한 I/O가 비동기적으로 이뤄지도록 합니다. defaults : rw, suid, dev, exec, auto, nouser, async를 기본 옵션으로 사용합니다. nosuid : SetUID, SetGID를 무시하게 됩니다. suid : SetUID, SetGID가 효력을 발휘할 수 있게 해줍니다. remount : 이미 마운트된 파일 시스템을 다시 마운트 합니다.

Mount 명령어 사용 형식

Mount 명령어의 사용 방법은 다음과 같습니다.

-o 옵션의 하위 옵션

옵션	설명	옵션	설명
rw	읽기/쓰기 모드로 마운트.	nodev	장치 파일을 무시.
ro	읽기 전용 모드로 마운트.	loop	파일을 디스크 이미지로 마운트.
noexec	실행 파일을 실행하지 않음.	defaults	기본 옵션(rw, suid, dev, exec, auto, nouser, async).
nosuid	SUID 비트를 무시.		

사용 예시

예시	설명
<code>sudo mount /dev/sda1 /mnt/mydisk</code>	/dev/sda1을 /mnt/mydisk에 마운트
<code>sudo mount -t ext4 /dev/sda1 /mnt/mydisk</code>	ext4 파일 시스템을 마운트
<code>sudo mount -o ro /dev/sda1 /mnt/mydisk</code>	읽기 전용 모드로 마운트
<code>sudo mount -t nfs 192.168.1.100/exported/path /mnt/nfs</code>	NFS 서버의 공유 디렉토리 마운트
<code>sudo mount -o loop /path/to/image.iso /mnt/iso</code>	ISO 이미지 파일을 마운트
<code>sudo mount -a</code>	/etc/fstab 파일에 정의된 모든 파일 시스템을 마운트
<code>mount -l</code>	UUID와 함께 마운트된 파일 시스템 목록 보기

/etc/fstab 파일

파일 시스템 마운트 정보를 저장하는 중요한 구성 파일입니다.

특징

부팅 시 자동으로 파일 시스템을 마운트하기 위해 사용되며, 총 6개의 필드로 구성이 되어 있습니다.

각 줄은 파일 시스템, 마운트 지점, 파일 시스템 유형, 마운트 옵션, 덤프 설정, 파일 시스템 검사 순서를 정의합니다.

사용 구문

<파일 시스템> <마운트 지점> <파일 시스템 유형> <옵션> <덤프> <패스>

사용 예시

```
UUID=ee0d397f-fa30-46b6-887d-24385c7dd351 /data/ xfs defaults,nofail 0 2
```

사용 옵션

옵션	설명	예시
<파일 시스템>	마운트할 장치나 파일 시스템의 경로입니다.	/dev/sda1, UUID, LABEL 등
<마운트 지점>	파일 시스템을 마운트할 디렉토리입니다.	/, /home, /mnt/data 등
<유형>	파일 시스템의 유형입니다.	ext4, swap, nfs, vfat 등
<옵션>	파일 시스템 마운트 옵션입니다.	defaults, ro, rw, noexec, nosuid, nodev 등
<덤프>	덤프 유틸리티에 의해 백업될지 여부를 나타냅니다.	0은 백업하지 않음을, 1은 백업함을 의미합니다.
<패스>	부팅 시 파일 시스템 검사를 할 순서입니다.	루트 파일 시스템은 1, 다른 파일 시스템은 2, 검사를 하지 않으려면 0으로 설정합니다.

`df(disk free)

디스크 공간 사용 현황을 확인할 때 사용하는 명령어입니다.

현재 시스템에 마운트된 파일 시스템의 전체 용량, 사용된 공간, 남은 공간 등을 표시합니다.

시스템의 디스크 공간을 모니터링하고 관리하는 데 사용되며, 용량이 부족한 파일 시스템을 식별하고 디스크 사용량을 확인하는 데 도움이 됩니다.

주로 서버 관리, 디스크 상태 점검 등에 유용합니다.

사용 예시

```
df -hT /dev/xvda1
```

Filesystem	Type	Size	Used	Avail	Use%	Mounted on
/dev/xvda1	xf	8.0G	2.1G	5.9G	27%	/

사용 옵션

옵션	설명
-h(human-readable)	출력 결과를 사람이 읽기 쉬운 단위(예: KB, MB, GB)로 표시합니다.
-a(all)	모든 파일 시스템을 표시합니다. 기본적으로는 사용되지 않는 파일 시스템은 생략됩니다.
-T(type)	파일 시스템의 타입(예: ext4, tmpfs)을 표시합니다.
--total	전체 파일 시스템의 합계를 출력합니다.
-i 또는 --inodes	디스크 공간 대신 아이노드(inode) 사용량을 표시합니다.

df(disk free)

디스크 공간 사용 현황을 확인할 때 사용하는 명령어입니다.

주요 출력 정보

항목	설명
Filesystem	파일 시스템 장치 이름
Type	파일 시스템 종류
Size	전체 디스크 용량
Used	사용 중인 용량
Avail	사용 가능한 용량
Use%	사용률
Mounted on	마운트된 경로

`lsblk 명령어

블록 장치들의 이름, 크기, 파일 시스템 유형, 마운트 지점(Point) 등을 출력하는 데 사용됩니다.

블록 장치에 대한 정보를 나열하는 것이 주된 목적이므로 "List Block Devices"의 약어로 lsblk가 사용됩니다.

블록 장치(예: 하드 드라이브, SSD, USB 드라이브 등)들의 이름, 크기, 파일 시스템 유형, 마운트 지점(Point) 등을 출력하는 데 사용됩니다.

또한, 디스크와 관련된 파티션 및 속성을 확인하는 데 유용합니다.

주요 출력 정보

항목	설명
NAME	디바이스 이름(예: sda, sdb1 등)
MAJ:MIN	커널 장치의 메이저와 마이너 디바이스 번호
RM	디바이스가 이동식인지 여부(0 = 고정, 1 = 이동식(USB))
SIZE	디스크/파티션의 크기
RO	읽기 전용 여부(0 = 읽기/쓰기 가능, 1 = 읽기 전용)
TYPE	디바이스 유형(디스크, 파티션, 루프 장치 등)
MOUNTPOINT	디바이스가 마운트된 위치

`lsblk 명령어

디스크와 스토리지 디바이스의 정보를 확인할 수 있는 명령어입니다.

사용 옵션

옵션	설명
-a, --all	마운트되지 않은 장치까지 포함한 모든 디바이스를 표시합니다.
-f	파일 시스템 정보 표시.
-d	디스크만 표시(파티션 제외).
-n	출력에서 헤더 생략.
-o, --output list	지정된 열만 표시합니다.(예: NAME,SIZE,MOUNTPOINT)
-p	전체 경로(절대 경로) 형식으로 디바이스 이름 표시.

사용 예시

```
lsblk
NAME      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
xvda      202:0    0   8G  0 disk
└─xvda1   202:1    0   8G  0 part /
```

예시	설명
lsblk	시스템에서 사용 가능한 모든 블록 장치에 대한 정보를 표시합니다. 블록 장치의 이름, 크기, 타입, 마운트 포인트 등의 정보를 확인할 수 있습니다.
lsblk -o NAME,MOUNTPOINT	마운트된 디스크와 파티션의 이름과 마운트 포인트만 표시합니다.

UUID

시스템에서 자원을 중복 없이 식별하기 위한 고유 식별자입니다.

UUID(Universally Unique Identifier)는 전 세계적으로 유일하도록 생성되는 식별자를 의미합니다.

UUID

128비트(16바이트) 값
보통 32자리 16진수 + 하이픈 4개
충돌 가능성이 사실상 0에 가까움

UUID의 구조

xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx

총 36자(하이픈 포함)
내부적으로는 시간, 난수, MAC 주소 등을 조합해 생성

필요성

/dev/sda1, /dev/xvda1 등의 디바이스의 이름은 부팅 순서, 디스크 추가에 따라 이름이 바뀔 수 있습니다.

UUID의 장점

절대 변하지 않음
어떤 환경에서도 동일한 장치를 정확히 식별
그래서 /etc/fstab 파일에서는 UUID 사용이 표준입니다.

UUID의 사용처

분야	사용 예시
리눅스	파일 시스템 식별
클라우드	볼륨, 리소스 ID
DB	기본 키(UUID PK)
분산 시스템	노드 · 요청 식별
웹	세션 ID, 토큰

blkid 명령어

시스템에 연결된 블록 장치(Block Device)의 정보를 출력하는 데 사용됩니다.

시스템에 연결된 디스크 파티션과 블록 장치(Block Device)의 정보를 출력하는 데 사용됩니다.

이 명령은 주로 파일 시스템 타입, UUID(Universally Unique Identifier), LABEL(레이블) 등의 정보를 확인할 때 유용합니다.

사용 구문

```
blkid [옵션] [장치명]
```

사용 예시

```
blkid
```

```
/dev/xvda1: LABEL="/" UUID="57c85852-7b96-4c8f-be4d-39efbf1ce53f" TYPE="xfs" PARTLABEL="Linux" PARTUUID="6ce8db92-d927-4730-88d1-a47a489fdb47"
```

사용 옵션

옵션	설명
-o list	출력 형식을 표 형태로 보여줌
-o value	키/값 형태로 출력
-t <유형> 또는 --match-type <유형>	특정 유형의 블록 장치만 표시(키/값 조건으로 필터링)합니다. 예를 들어, -t ext4는 ext4 파일 시스템을 가진 장치만 표시합니다.
-p	특정 파일 시스템의 데이터 확인
-u	캐시를 사용하지 않고 디바이스 정보를 재검사

dd 명령어

원하는 크기의 임시 데이터를 인위적으로 만들어내는 저수준 복사 명령어입니다.

주요 용도

테스트용 파일 생성(실습에서 가장 많이 사용)
디스크 용량 테스트, du 실습용
디스크 전체를 이미지 파일로 백업
디스크를 디스크로 그대로 복사
디스크를 0으로 덮어쓰기

기본 구조

옵션	의미
if	input file(읽을 대상)
of	output file(쓸 대상)
bs	block size
count	블록 반복 횟수

사용 예시

```
sudo dd if=/dev/zero of=/mnt/testdisk/data/testfile bs=1M count=100
```

옵션 해석

if=/dev/zero : 0으로 채워진 가짜 데이터

of=.../testfile : 생성될 파일 이름

bs=1M : 한 번에 1MB씩 쓰기

count=100 : 100번 쓰기

du 명령어

디렉토리나 파일이 실제로 차지하고 있는 디스크 사용량을 확인하는 명령어입니다.

du(disk usage)는 파일 시스템에 실제로 할당된 블록 기준으로 특정 디렉토리, 하위 디렉토리별 용량 분석에 사용되는 명령어입니다.

사용 구문

du [옵션] [파일 또는 디렉토리]

사용 옵션

옵션	설명
-h	사람이 읽기 쉬운 단위(K, M, G)
-s	합계(Summary)만 출력
-a	파일까지 포함하여 출력
--max-depth=N	하위 디렉토리 깊이 제한
-c	전체 합계 추가 출력
-x	다른 파일 시스템 제외
-L	심볼릭 링크 대상 기준으로 계산

리눅스(Linux)

메모리 & 프로세스 관리



≡ 메모리 & 프로세스 관리

≡ `free` 명령어

시스템의 메모리 사용 현황을 확인하는 명령어입니다.

물리 메모리(RAM), 스왑 메모리(Swap), 사용 중 / 여유 / 캐시 상태를 한눈에 보여 주는 명령어 입니다.

사용 형식

free [옵션]

사용 예시

예시	설명
free	기본 형식으로 메모리의 정보를 보여 줍니다.
free -h	free 명령어와 동일하지만 사람이 읽기 쉽게 보여 줍니다. 예를 들어서 수치가 6977824이면 6.7Gi 등의 형식으로 보여 줍니다.

≡ 메모리 & 프로세스 관리

≡ `ps` 명령어

현재 실행 중인 모든 프로세스의 상세 정보를 보여줍니다.

출력 컬럼

컬럼	의미
UID	실행 사용자
PID	프로세스 구분을 위한 고유한 ID
PPID	부모의 프로세스 ID
C	CPU 사용률
STIME	시작 시간
TTY	해당 프로세스의 제어 터미널
TIME	CPU 사용 시간
CMD	실행 명령

☰ 메모리 & 프로세스 관리

☰ `pstree` 명령어

메모리 & 프로세스 관리

메모리 & 프로세스 관리

사용 형식

메모리 & 프로세스 관리

사용 예시

예시	설명

☰ 메모리 & 프로세스 관리

☰ `top` 명령어

메모리 & 프로세스 관리

메모리 & 프로세스 관리

사용 형식

메모리 & 프로세스 관리

사용 예시

예시

설명

☰ 메모리 & 프로세스 관리

☰ `kill` 명령어

메모리 & 프로세스 관리

메모리 & 프로세스 관리

사용 형식

메모리 & 프로세스 관리

사용 예시

예시

설명

☰ 메모리 & 프로세스 관리

☰ `killall` 명령어

메모리 & 프로세스 관리

메모리 & 프로세스 관리

사용 형식

메모리 & 프로세스 관리

사용 예시

예시

설명

☰ 메모리 & 프로세스 관리

☰ `uptime` 명령어

메모리 & 프로세스 관리

메모리 & 프로세스 관리

사용 형식

메모리 & 프로세스 관리

사용 예시

예시

설명

리눅스(Linux)

Linux Command



`sed` 명령어

파일 내의 특정 문자열을 찾아 다른 문자열로 대체하는 작업을 수행합니다.

`sed`는 stream editor의 줄인 말로써, 스트림 편집기 프로그램입니다.

이 명령어를 사용하여 파일의 내용을 실시간으로 수정하거나, 파일 내용을 출력하여 다른 파일로 저장하는 데 유용합니다.

기본적으로 텍스트 스트림을 처리하며, 텍스트 변경, 삭제, 삽입, 치환 등을 할 수 있습니다.

사용 구문

```
sed [옵션] '명령' 파일
```

사용 옵션

옵션	설명
-i	파일을 직접 수정합니다. 이 옵션을 사용하면 `sed` 명령어가 파일을 직접 수정하고, 수정된 내용을 파일에 저장합니다. (백업 옵션도 함께 사용할 수 있습니다.)
-e	여러 개의 명령을 한 번에 실행할 수 있습니다.
-n	출력을 억제하고, 명령어에서 지정한 내용만 출력합니다.
-r 또는 -E	확장 정규 표현식을 사용하여 패턴을 작성할 수 있습니다.

`Shell Script

리눅스 셸(Shell)에서 실행할 명령어들을 모아놓은 텍스트 파일입니다.

일반적으로 Bash(Bourne Again Shell)에서 실행되며, 파일 안에 명령어, 변수, 조건문, 반복문 등을 포함하여 자동화 작업을 수행할 수 있습니다.

주요 특징

항목	설명
자동화	반복적인 작업을 자동으로 처리할 수 있습니다.
텍스트 기반	모든 명령은 텍스트 파일로 작성됩니다.
실행 방식	실행 전, chmod 명령어를 사용하여 스크립트 파일에 실행 권한을 부여해야 합니다.
주석 지원	`#` 기호로 시작하는 줄은 주석으로 처리됩니다.
확장성	스크립트에서 다른 프로그램이나 유틸리티를 호출할 수 있습니다.

실행 방법

항목	설명
파일 생성	vi hello.sh # 내용을 입력 후 저장.
실행 권한 부여	chmod +x hello.sh
실행	./hello.sh

리눅스(Linux)

Nat/Bridge 설정



☰ NAT 모드와 Bridge 모드

☰ NAT 모드와 Bridge 모드

NAT 모드와 Bridge 모드에 대하여 학습합니다.

NAT 모드

Network Address Translation

가상 머신이 호스트 PC를 통해서만 외부 네트워크에 접근

가상 머신은 사설 IP를 사용

외부에서는 가상 머신이 보이지 않음

Bridge(브리지) 모드

가상 머신이 호스트와 동일한 네트워크에 직접 연결

가상 머신이 실제 네트워크의 IP를 받음

외부에서 직접 접근 가능

≡ 차이점

NAT 모드와 Bridge 모드의 차이점에 대하여 살펴 봅니다.

구분	Nat 모드	Bridge 모드
IP 주소	가상 사설 IP	실제 네트워크 IP
외부 접근	불가능(기본)	가능
인터넷 연결	매우 쉬움	가능
설정 난이도	쉬움	보통
보안성	높음	낮음
서버 실습	X 부적합	O 적합
실제 PC처럼 사용	X	O

☰ NAT 모드와 Bridge 모드

☰ NAT 모드와 Bridge 모드

NAT 모드의 장단점에 대하여 살펴 봅니다

장점

설정이 가장 간단
인터넷 연결 즉시 가능
외부 접근 차단 → 보안 안전
공공 Wi-Fi에서도 잘 동작

단점

외부에서 VM 접속 불가
서버 실습(웹, DB, SSH)에 불편
포트 포워딩 설정 필요

☰ NAT 모드와 Bridge 모드

☰ NAT 모드와 Bridge 모드

Bridge 모드의 장단점에 대하여 살펴 봅니다

장점

VM을 실제 서버처럼 사용 가능
SSH, 웹 서버, DB 접속 실습에 최적
팀 프로젝트, 네트워크 실습에 유리

단점

네트워크 환경에 따라 연결 실패 가능
IP 충돌 위험
외부 공격 노출 가능(방화벽 필수)

☰ NAT 모드와 Bridge 모드

☰ NAT 모드 사용 시점

일반적으로 NAT 모드를 사용하는 상황은 다음과 같습니다.

NAT 모드

리눅스를 처음 배우는 단계

패키지 설치, 개발 테스트

단순히 인터넷만 필요할 때

노트북 이동이 잦을 때

추천 예:

리눅스 기초 수업, 개인 연습용 VM

≡ NAT 모드와 Bridge 모드

≡ Bridge 모드 사용 시점

일반적으로 Bridge 모드를 사용하는 상황은 다음과 같습니다.

Bridge 모드

SSH 접속 실습

웹 서버(Apache/Nginx) 실습

Spring Boot / Docker / Kubernetes 서버 실습

실제 서버 환경 재현

추천 예:

서버 운영 수업, 클라우드 실습, 팀 프로젝트

≡ NAT 모드와 Bridge 모드

≡ Bridge 모드 IP 주소 고정

Bridge 모드에서 IP 주소를 고정시키는 방법에 대하여 살펴 봅니다.

Bridge 모드에서는 호스트가 아니라, 실제 네트워크(공유기/스위치)의 DHCP 정책을 그대로 따르기 때문에 아무 설정 없이 두면 IP가 자주 바뀌는 것이 정상입니다. IP 주소를 항상 고정하려면 다음과 같이 설정하면 됩니다.

ipconfig

Step01. Window 확인

Windows IP 구성

이더넷 어댑터 이더넷:

연결별 DNS 접미사. . . . : kornet

링크-로컬 IPv6 주소 : fe80::d834:680e:c8ca:eb72%3

IPv4 주소 : 175.211.43.221

서브넷 마스크 : 255.255.255.0

기본 게이트웨이 : 175.211.43.254

이더넷 어댑터 이더넷 2:

연결별 DNS 접미사. . . . :

링크-로컬 IPv6 주소 : fe80::6b0a:46f4:f5d3:c3e3%12

IPv4 주소 : 192.168.56.1

서브넷 마스크 : 255.255.255.0

기본 게이트웨이 :

여기서 실제 인터넷 연결(진짜 NIC)을 찾는 방법은 '기본 게이트웨이' 항목입니다. 즉, '이더넷 어댑터 이더넷'이 진짜 NIC입니다.

여기서 'IPv4 주소'와 '기본 게이트웨이' 항목을 확인합니다.

이더넷 어댑터 이더넷:

IPv4 주소 : 175.211.43.221

기본 게이트웨이 : 175.211.43.254

☰ NAT 모드와 Bridge 모드

☰ Bridge 모드 IP 주소 고정

Bridge 모드에서 IP 주소를 고정시키는 방법에 대하여 살펴 봅니다.

01-netcfg.yaml 은 Ubuntu에서 네트워크를 "정의(설계)"하는 설정 파일입니다.

즉, 이파일은 컴퓨터가 어떤 IP를 쓰고, 어떤 길(게이트웨이)로 나갈지 정하는 설계도 파일입니다.

```
sudo vi /etc/netplan/01-netcfg.yaml
```

```
network:
```

```
  version: 2
```

```
  renderer: networkd
```

```
  ethernets:
```

```
    enp0s3:
```

```
      dhcp4: false
```

```
      addresses:
```

```
        - 175.211.43.10/24
```

```
      routes:
```

```
        - to: default
```

```
          via: 175.211.43.254
```

```
      nameservers:
```

```
        addresses:
```

```
          - 8.8.8.8
```

```
          - 1.1.1.1
```

Step02. 네트워크 설정 파일

```
# 파일 퍼미션 설정
```

```
sudo chmod 600 /etc/netplan/01-netcfg.yaml
```

```
# 설정 파일 적용시키기
```

```
sudo netplan apply
```

```
# IP 주소 다시 확인
```

```
ip addr
```


리눅스(Linux)

유용한 명령어



메이븐(maven)

다음은 메이븐에서 사용할 수 있는 유용한 명령어입니다.

관련 명령어

명령어	용도
mvn -version	Maven 설치 확인
mvn clean	이전 빌드 결과 초기화, target/ 디렉토리 삭제
mvn compile	Java 소스 코드 컴파일, target/classes 생성
mvn test	테스트 실행
mvn package	컴파일 + 테스트 + JAR/WAR 파일 생성, 결과물은 target/*.jar
mvn install	로컬 저장소 설치
mvn spring-boot:run	Spring Boot 실행
mvn package -DskipTests	테스트 생략 빌드

메이븐(maven)

메이븐으로 컴파일시 발생할 수 있는 오류 내용입니다.

Bridge 모드

[INFO] BUILD FAILURE

... 중략

[ERROR] Failed to execute goal ng: error: **release version 17 not supported**

조치 사항

```
sudo apt install -y openjdk-17-jdk
```

java 실행 파일이 어떤 JDK/JRE 버전을 사용할지 선택합니다.

```
sudo update-alternatives --config java
```

자바 컴파일러가 어떤 JDK 버전을 사용할지 선택합니다.

```
sudo update-alternatives --config javac
```

```
mvn clean package -DskipTests
```

```
ls target/docker_spring_mysql.jar
```

MySQL

다음은 MySQL에서 사용할 수 있는 유용한 명령어입니다.

서비스 명령어

명령어	용도
<code>sudo systemctl status mysql</code>	MySQL 서비스 상태 확인
<code>sudo systemctl start mysql</code>	MySQL 시작
<code>sudo systemctl stop mysql</code>	MySQL 중지
<code>sudo systemctl restart mysql</code>	MySQL 다시 시작

접속 명령어

명령어	용도
기본 접속	<code>mysql -u root -p</code> 비밀 번호를 입력(눈에 보이지 않음) 후 접속하기
특정 DB로 접속	<code>mysql -u user01 -p dbname</code>
원격 DB 접속	<code>mysql -u user01 -p -h 192.168.0.10 -P 3306</code>

MySQL

다음은 MySQL에서 사용할 수 있는 유용한 명령어입니다.

Database 관련

명령어	용도
DB 생성	CREATE DATABASE testdb;
DB 선택	USE testdb;
DB 삭제	DROP DATABASE testdb;
DB 목록 보기	SHOW DATABASES;

Table 관련

명령어	용도
테이블 목록	SHOW TABLES;
테이블 구조 확인	DESC member;
테이블 생성	CREATE TABLE member(id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(50), email VARCHAR(100));
테이블 삭제	DROP TABLE member;

MySQL

다음은 MySQL에서 사용할 수 있는 유용한 명령어입니다.

사용자 & 권한 관리

명령어	용도
사용자 생성	CREATE USER 'user01'@'%' IDENTIFIED BY '1234';
권한 부여	GRANT ALL PRIVILEGES ON testdb.* TO 'user01'@'%';
권한 반영	FLUSH PRIVILEGES;
사용자 삭제	DROP USER 'user01'@'%';

☰ `MySQL

다음은 MySQL에서 사용

사용자 & 권한 관리

명령어

7☐ 파일 백업 & 복구(Linux + MySQL)

DB 백업

```
mysqldump -u root -p testdb > testdb.sql
```

DB 복구

```
mysql -u root -p testdb < testdb.sql
```

8☐ MySQL 내부 정보 확인용 명령어

현재 사용자

```
SELECT USER();
```

현재 DB

```
SELECT DATABASE();
```

MySQL 버전

```
SELECT VERSION();
```

9☐ MySQL 종료

```
EXIT;
```